LECTURE · LINUX FOR CYBERSECURITY

# The 50 Most-Used Linux Commands

for security analysts, sysadmins and blue/red-team learners — every command with a hands-on terminal practice.

50 commands · 6 modules · practice tasks on every slide



BEFORE WE START

# Your lab & the rules

- **Practise on systems you own** — a Kali VM plus one or two target VMs on a host-only network (e.g. 192.168.56.0/24).
- **Scanning or testing other people's systems without written permission is illegal** in most countries. scanme.nmap.org is one host that explicitly allows light scanning.
- **Take snapshots** before you experiment, so you can always roll back.
- Output in these slides is **example output** from a lab: your numbers will differ.

lab setup

```
# typical lab layout
kali@kali:~$ ip -br a
eth0  UP  192.168.56.101/24   # Kali
# target VM: 192.168.56.10 (lab-web)
kali@kali:~$ ping -c 1 192.168.56.10
64 bytes from 192.168.56.10: ttl=64 time=0.4 ms
# ready.
kali@kali:~$
```

AGENDA

# Six modules, fifty commands

## Files & Navigation 9

`pwd` · `ls` · `cd` · `cat` · `cp` · `mv` · `rm` · `find` · `file`

## Text Processing 10

`grep` · `awk` · `sed` · `cut` · `sort` · `uniq` · `wc` · `tail` · `strings` · `xxd`

## Users & Permissions 6

`whoami` · `id` · `sudo` · `su` · `chmod` · `chown`

## Processes & System 8

`ps` · `top` · `kill` · `lsof` · `uname` · `systemctl` · `journalctl` · `crontab`

## Networking 12

`ip` · `ss` · `ping` · `traceroute` · `dig` · `curl` · `wget` · `nc` · `ssh` · `scp` · `nmap` · `tcpdump`

## Security & Archives 5

`iptables` · `tar` · `base64` · `sha256sum` · `openssl`

MODULE 1 OF 6

# Files & Navigation

`pwd` `ls` `cd` `cat` `cp` `mv` `rm` `find` `file`

01/50 FILES & NAVIGATION

# pwd

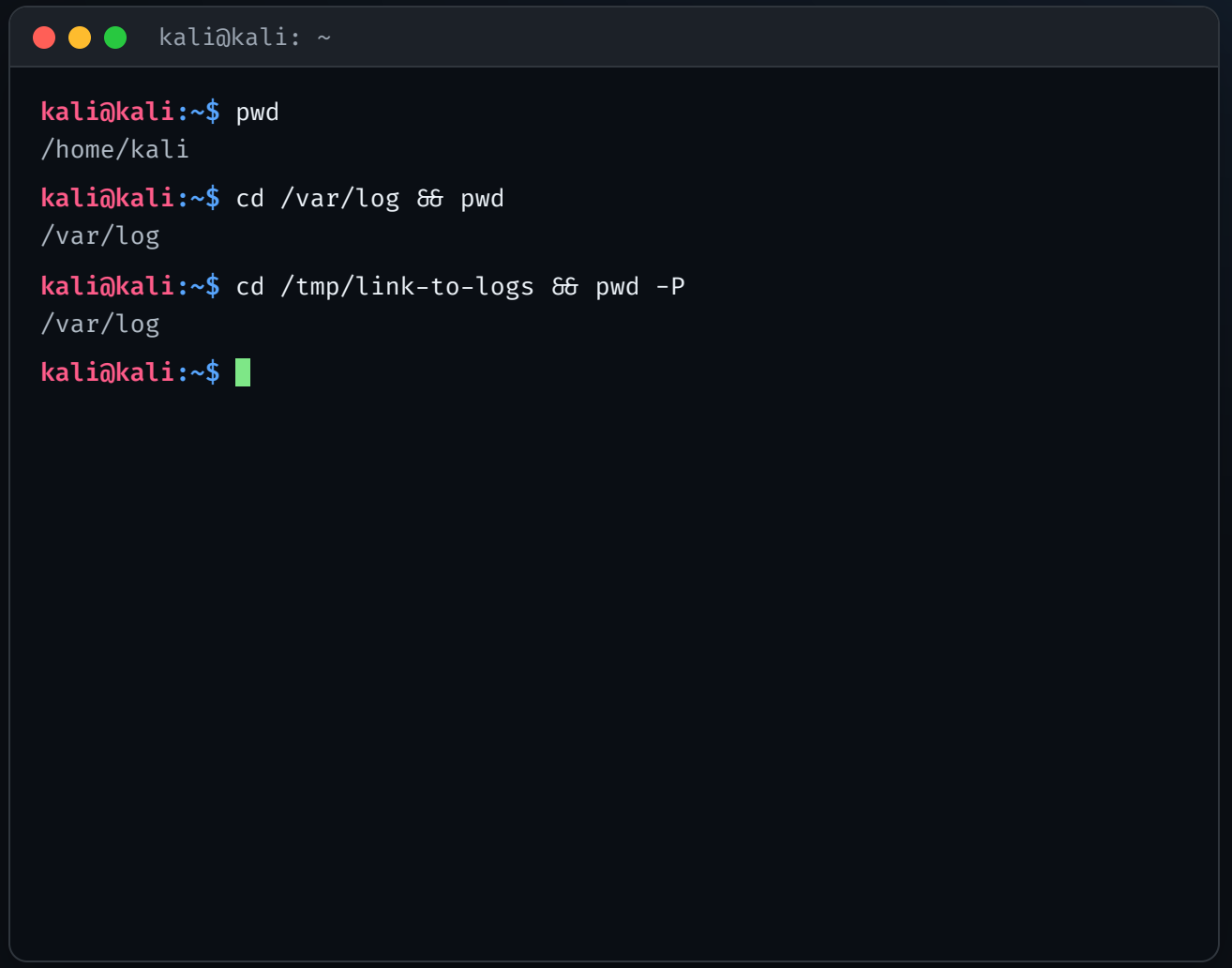
Prints the full path of the directory you are currently in.

```
pwd [-P]
```

| | |
|---|---|
| `-P` | show the physical path (resolve symlinks) |
| `-L` | show the logical path (default) |

**SECURITY USE**

Always confirm where you are before running commands that change or delete files during an investigation.




▶ **Practice** Make a symlink to /etc in your home folder, cd into it, and compare pwd with pwd -P.

02/50 FILES & NAVIGATION

# ls

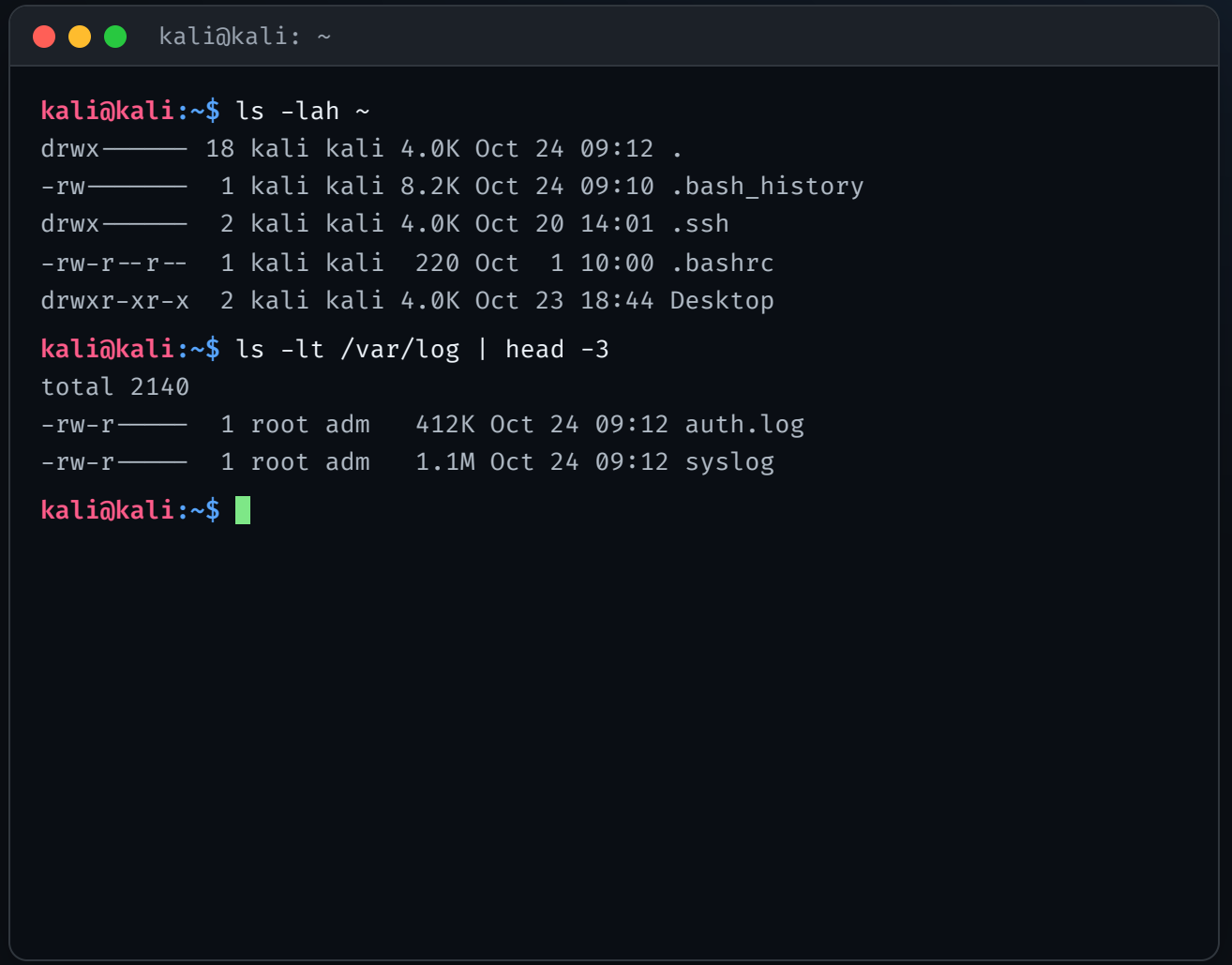
Lists directory contents, including hidden files and permissions.

```
ls [options] [path]
```

| | |
|---|---|
| `-l` | long format: permissions, owner, size, date |
| `-a` | include hidden (dot) files |
| `-h` | human-readable sizes |
| `-t` | sort by modification time |

**SECURITY USE**

Hidden files and odd permissions are often the first clue when auditing a home folder or web root.




▶ **Practice** List /etc sorted by modification time. Which config file changed most recently?

# cd

Changes the current working directory.

```
cd [directory]
```

| | |
|---|---|
| `cd ~` | go to your home directory |
| `cd -` | jump back to the previous directory |
| `cd ..` | go up one level |
| `cd /` | go to the filesystem root |

**SECURITY USE**

Moving quickly between /var/log, /etc and /tmp is the backbone of every triage session.




▶ **Practice** From your home folder, reach /var/log/apt using one absolute and one relative path.

# cat

Concatenates files and prints them to the terminal.

```
cat [options] file ...
```

| | |
|---|---|
| `-n` | number every line |
| `-A` | show hidden characters (tabs, line ends) |
| `-s` | squeeze repeated blank lines |

**SECURITY USE**

Quickly read configs such as /etc/passwd or /etc/hosts, and reveal hidden characters with -A.

kali@kali: ~

```
kali@kali:~$ cat /etc/hosts
127.0.0.1   localhost
127.0.1.1   kali
192.168.56.10 lab-web.local
kali@kali:~$ cat -n /etc/passwd | tail -3
  46  kali:x:1000:1000:Kali,,,:/home/kali:/usr/bin/zsh
  47  postgres:x:122:130::/var/lib/postgresql:/bin/bash
  48  analyst:x:1001:1001::/home/analyst:/bin/bash
kali@kali:~$
```

▶ **Practice** Use cat -n on /etc/passwd. How many accounts have a real login shell?

# cp

Copies files and directories.

```
cp [options] source dest
```

| | |
|---|---|
| `-r` | copy directories recursively |
| `-p` | preserve mode, owner and timestamps |
| `-a` | archive mode (recursive + preserve) |
| `-i` | ask before overwriting |

**SECURITY USE**

Preserve evidence: copy logs with -a so timestamps and ownership stay intact for analysis.




▶ **Practice** Copy /etc/ssh/sshd_config with and without -p and compare the timestamps with ls -l.

# mv

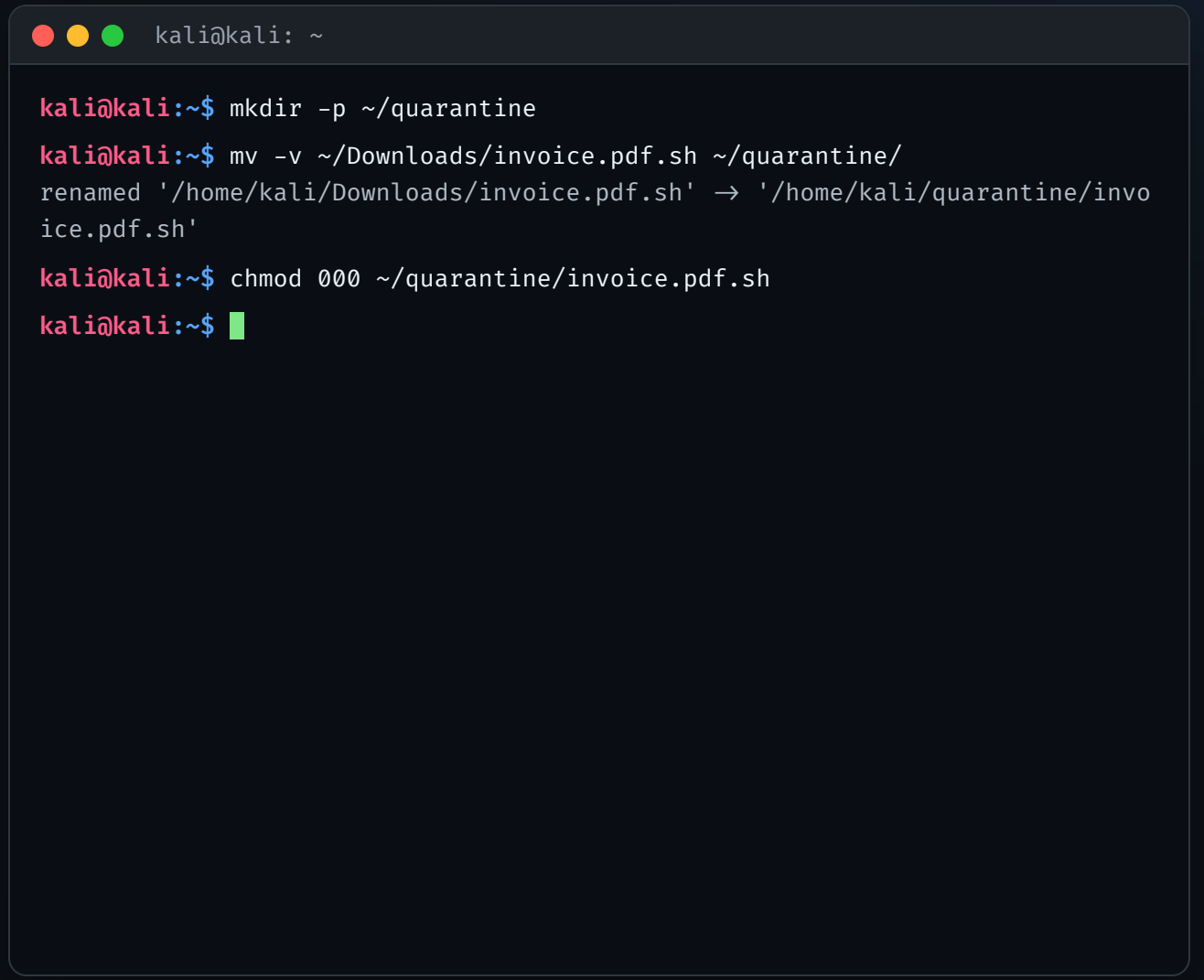
Moves or renames files and directories.

```
mv [options] source dest
```

| | |
|---|---|
| `-i` | prompt before overwrite |
| `-n` | never overwrite |
| `-v` | verbose: show what was moved |

**SECURITY USE**

Quarantine a suspicious file by moving it into an isolated folder instead of deleting it.




▶ **Practice** Create a test file, rename it with mv -v, then move it into a new folder.

# rm

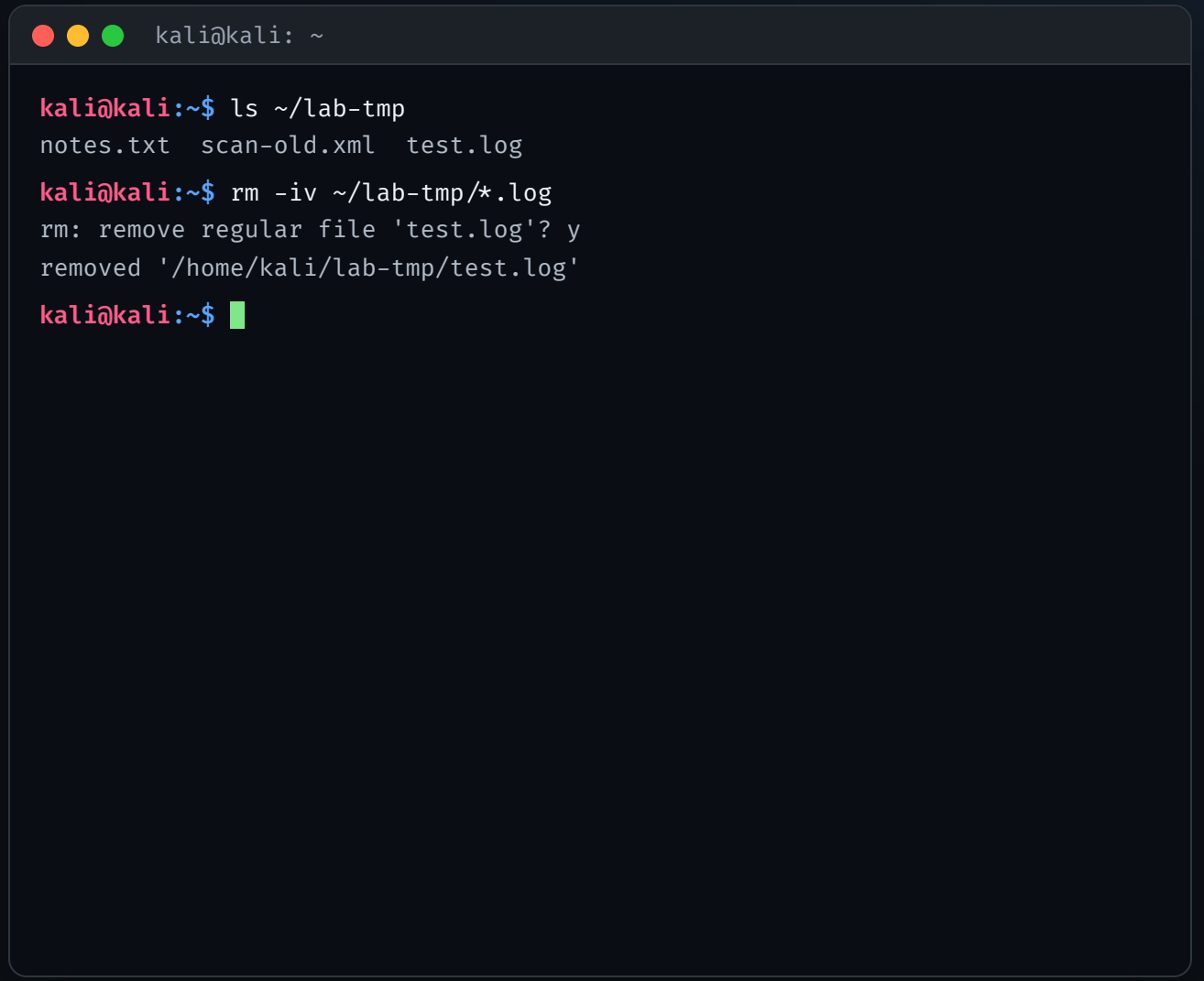
Removes files or directories. There is no recycle bin.

```
rm [options] file ...
```

| | |
|---|---|
| `-i` | ask before every removal |
| `-r` | remove directories recursively |
| `-f` | force, ignore missing files |
| `-v` | verbose |

**SECURITY USE**

Clean up lab artifacts safely. Double-check paths: rm -rf on the wrong path is unrecoverable.




▶ **Practice** Create three files in a test folder and remove only the .log files using rm -i.

# find

Searches the filesystem by name, type, size, owner, time or permission.

```
find [path] [tests] [actions]
```

| | |
|---|---|
| `-name` | match a filename pattern |
| `-perm` | match permissions (e.g. -4000 for SUID) |
| `-mtime` | modified N days ago |
| `-user` | owned by a user |

**SECURITY USE**

Auditors hunt for SUID binaries, world-writable files and recently changed files after an incident.

kali@kali: ~

```
kali@kali:~$ find / -perm -4000 -type f 2>/dev/null | head -4
/usr/bin/passwd
/usr/bin/sudo
/usr/bin/mount
/usr/bin/su
kali@kali:~$ find /etc -mtime -1 -type f 2>/dev/null
/etc/hosts
kali@kali:~$ find /tmp -perm -o+w -type f
/tmp/build.lock
kali@kali:~$
```

▶ **Practice** Find every file in /etc modified in the last 7 days, and every SUID file on your system.

09/50 FILES & NAVIGATION

# file

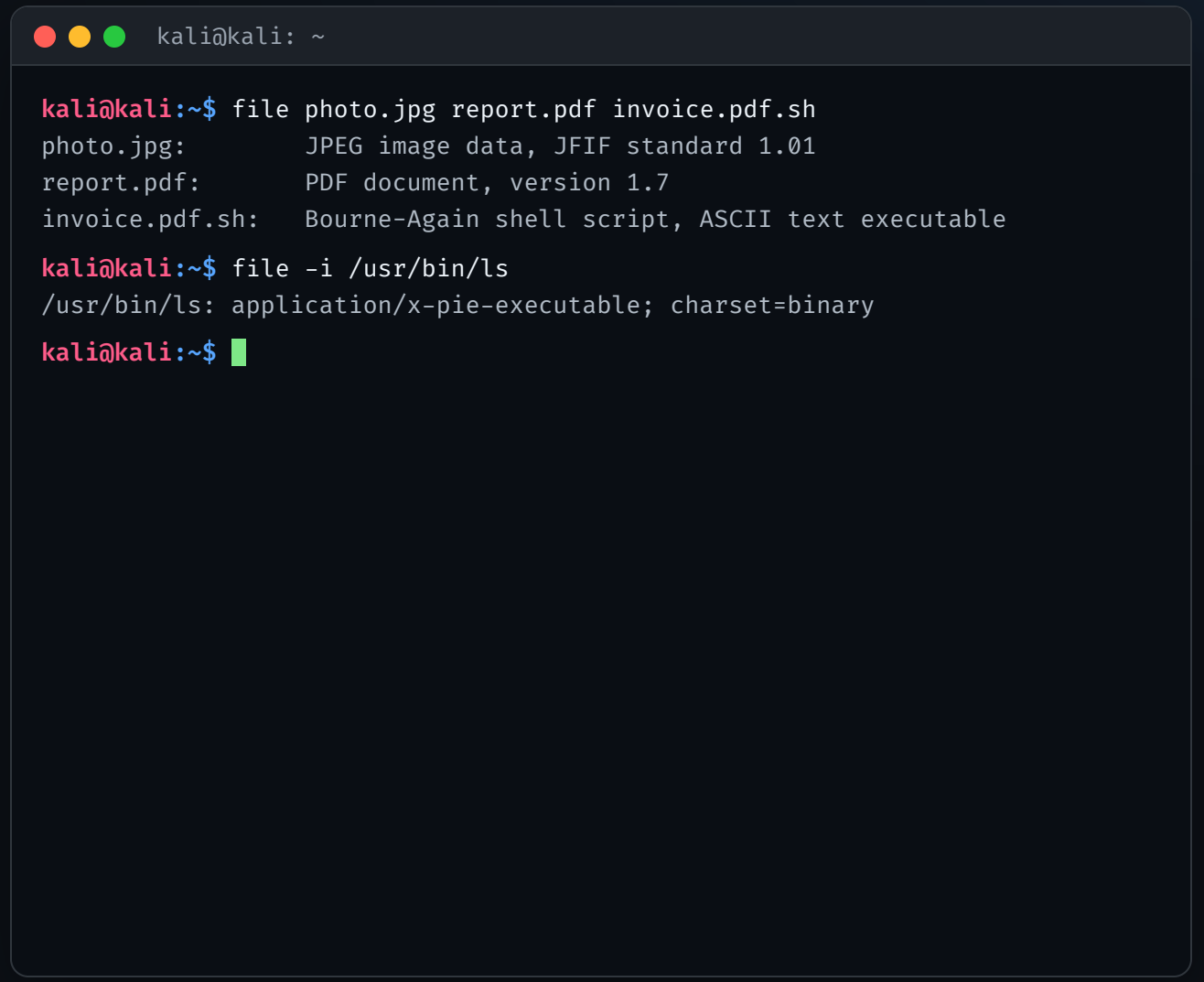
Identifies a file's real type by its content, not its extension.

```
file [options] file ...
```

| | |
|---|---|
| `-b` | brief: omit the filename |
| `-i` | print the MIME type |
| `-z` | look inside compressed files |

**SECURITY USE**

Extensions lie. 'photo.jpg' might really be an ELF executable or a script; file reveals it.




▶ **Practice** Rename a text file to image.png, then run file on it. What does it report?

MODULE 2 OF 6

# Text Processing

`grep` `awk` `sed` `cut` `sort` `uniq` `wc` `tail` `strings` `xxd`

# grep

Searches text for lines matching a pattern or regular expression.

```
grep [options] pattern [file ... ]
```

| | |
|---|---|
| `-i` | case-insensitive |
| `-r` | search directories recursively |
| `-v` | invert: show non-matching lines |
| `-E` | extended regex |
| `-c` | count matches |

**SECURITY USE**

The number one log-analysis tool: find failed logins, error codes or indicators of compromise.

kali@kali: ~

```
kali@kali:~$ sudo grep -c 'Failed password' /var/log/auth.log
37
kali@kali:~$ sudo grep 'Failed password' /var/log/auth.log | tail -2
Oct 24 08:51:02 kali sshd[2210]: Failed password for root from 192.168.56.23
port 50412
Oct 24 08:51:05 kali sshd[2210]: Failed password for root from 192.168.56.23
port 50418
kali@kali:~$ grep -rEi 'password|secret' ~/project/config
config/app.env:DB_PASSWORD=TODO_change_me
kali@kali:~$
```

▶ **Practice** Search /var/log/auth.log for 'Accepted' to list successful logins on your machine.

# awk

A pattern-scanning language for extracting and reshaping columns of text.

```
awk 'pattern { action }' file
```

| | |
|---|---|
| `-F` | set the field separator |
| `$1, $NF` | first field, last field |
| `NR` | current line number |

**SECURITY USE**

Pull specific columns out of logs, e.g. the source IP of every failed login.

kali@kali: ~

```
kali@kali:~$ awk -F: '$7 ~ /bash|zsh/ {print $1}' /etc/passwd
root
kali
postgres
analyst
kali@kali:~$ sudo grep 'Failed password' /var/log/auth.log | awk '{print $(N
F-3)}' | head -3
192.168.56.23
192.168.56.23
192.168.56.41
kali@kali:~$
```

▶ **Practice** Use awk to print each user's name and home directory from /etc/passwd.

# sed

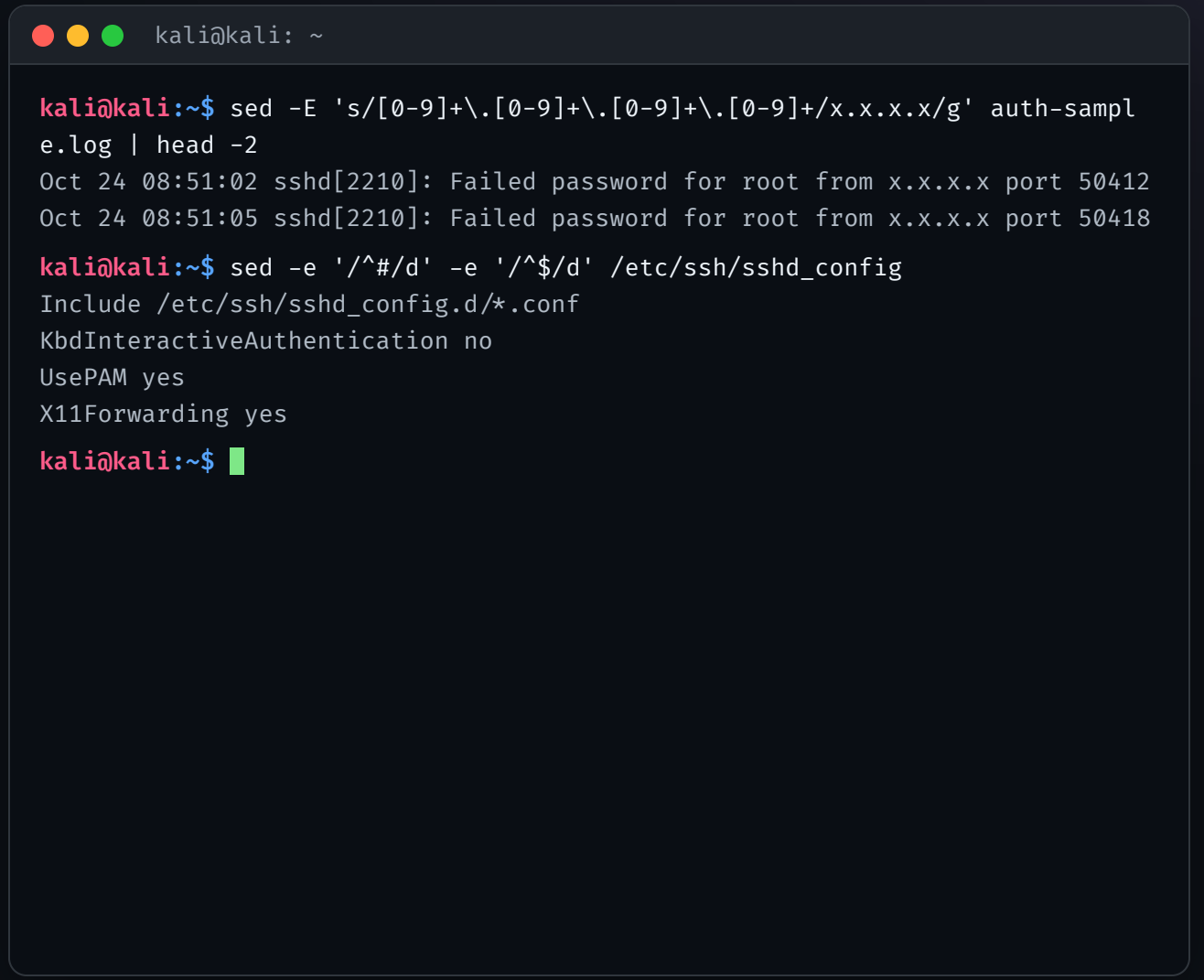
A stream editor: find-and-replace and line edits on text streams.

```
sed [options] 'script' file
```

| | |
|---|---|
| `s/a/b/g` | substitute a with b globally |
| `-n 'Np'` | print only line N |
| `-i` | edit the file in place |
| `/re/d` | delete matching lines |

**SECURITY USE**

Redact sensitive data before sharing logs, or strip comments to see the active config.




▶ **Practice** Show only the active (non-comment) lines of /etc/ssh/sshd_config.

# cut

Extracts sections (fields or characters) from each line.

```
cut [options] file
```

| | |
|---|---|
| `-d` | field delimiter |
| `-f` | field numbers to keep |
| `-c` | character positions to keep |

**SECURITY USE**

Fast column extraction from CSVs, /etc/passwd or exported logs.

kali@kali: ~

```
kali@kali:~$ cut -d: -f1,6 /etc/passwd | tail -3
kali:/home/kali
postgres:/var/lib/postgresql
analyst:/home/analyst
kali@kali:~$ cut -d, -f2 logins.csv | head -3
src_ip
192.168.56.23
192.168.56.41
kali@kali:~$
```

▶ **Practice** Use cut to list only the usernames from /etc/passwd.

# sort

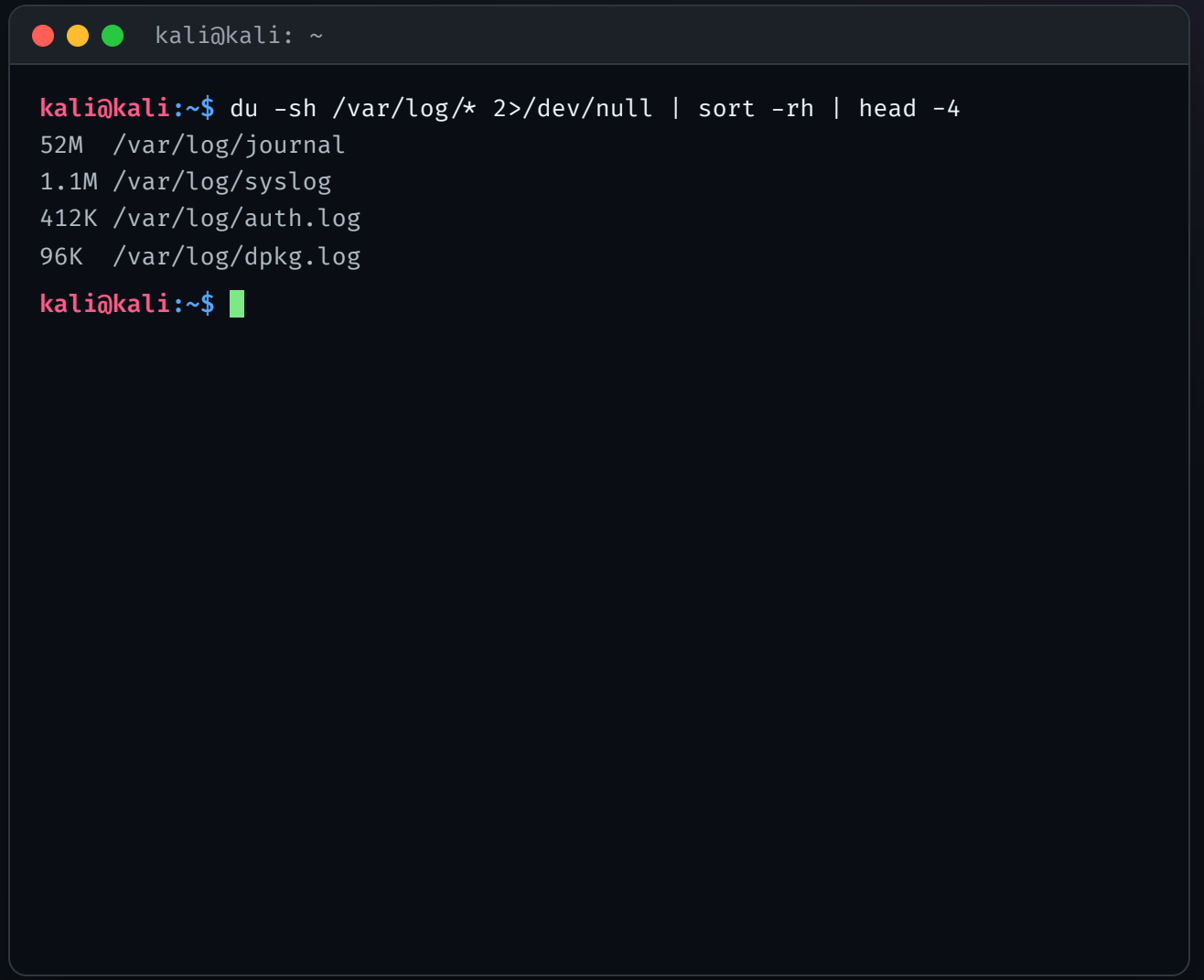
Sorts lines alphabetically, numerically or by a chosen column.

```
sort [options] file
```

| | |
|---|---|
| `-n` | numeric sort |
| `-r` | reverse order |
| `-k` | sort by a column |
| `-u` | unique lines only |
| `-h` | human sizes (1K, 2M) |

**SECURITY USE**

Combined with uniq, sort turns raw logs into ranked lists such as 'top attacking IPs'.




▶ **Practice** Find the five largest folders in your home directory using du and sort.

15/50 TEXT PROCESSING

# uniq

Filters or counts adjacent duplicate lines (sort first).

```
uniq [options] [file]
```

| | |
|---|---|
| `-c` | prefix lines with their count |
| `-d` | only show duplicated lines |
| `-u` | only show unique lines |

**SECURITY USE**

The classic 'who is hammering my SSH?' pipeline: grep, extract, sort, uniq -c, sort -rn.

kali@kali: ~

```
kali@kali:~$ sudo grep 'Failed password' /var/log/auth.log \
    | awk '{print $(NF-3)}' | sort | uniq -c | sort -rn
     29 192.168.56.23
      6 192.168.56.41
      2 192.168.56.7
kali@kali:~$
```

▶ **Practice** Build the pipeline above on your own auth.log. Which source has the most failures?

# wc

Counts lines, words and bytes.

```
wc [options] file
```

| | |
|---|---|
| `-l` | count lines |
| `-w` | count words |
| `-c` | count bytes |

**SECURITY USE**

Quantify events: how many failed logins, how many users, how many lines in a wordlist.

kali@kali: ~

```
kali@kali:~$ wc -l /etc/passwd
48 /etc/passwd
kali@kali:~$ sudo grep -c 'Accepted' /var/log/auth.log
5
kali@kali:~$ ls /usr/bin | wc -l
2431
kali@kali:~$
```

▶ **Practice** Count how many lines in /etc/group mention the 'sudo' group.

17/50 TEXT PROCESSING

# tail

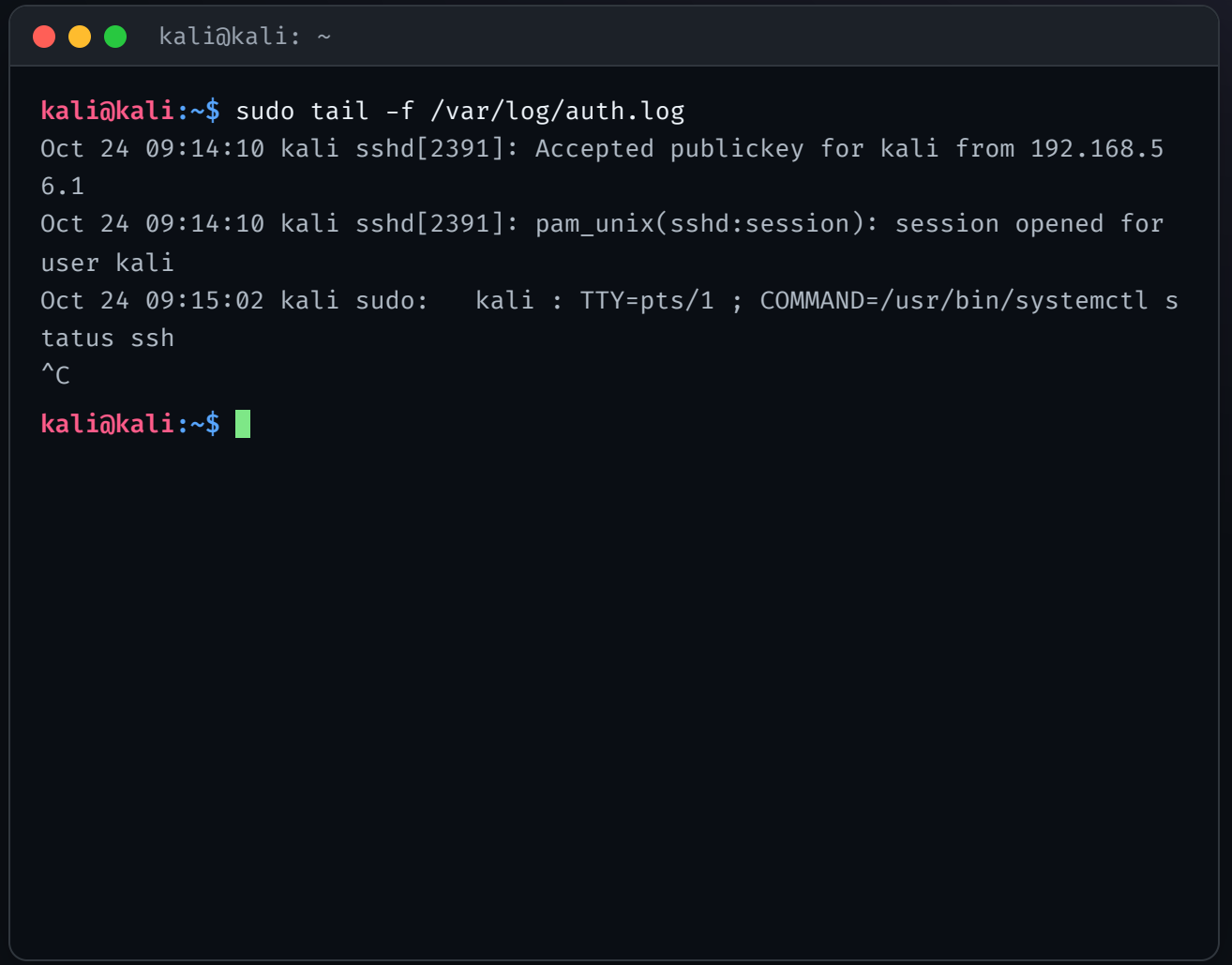
Shows the end of a file and can follow it live as it grows.

```
tail [options] file
```

| | |
|---|---|
| `-n N` | show the last N lines |
| `-f` | follow new lines in real time |
| `-F` | follow even if the file is rotated |

**SECURITY USE**

Watch authentication or web-server logs live while you test or while an incident unfolds.




▶ **Practice** Run tail -f on auth.log in one terminal and use sudo in another. Watch the entry appear.

# strings

Prints the readable text sequences hidden inside binary files.

```
strings [options] file
```

| | |
|---|---|
| `-n N` | minimum string length |
| `-a` | scan the whole file |
| `-e l` | 16-bit little-endian (Windows) strings |

**SECURITY USE**

First look at an unknown binary: embedded URLs, paths and messages reveal what it does.

kali@kali: ~

```
kali@kali:~$ strings -n 8 ./unknown-sample | grep -Ei 'http|/etc|/tmp'
http://update.example.com/check
/etc/cron.d/
/tmp/.cache_x
# indicators worth noting in the report
kali@kali:~$
```

▶ **Practice** Run strings on /usr/bin/ls and look for its usage/help text.

19/50 TEXT PROCESSING

# xxd

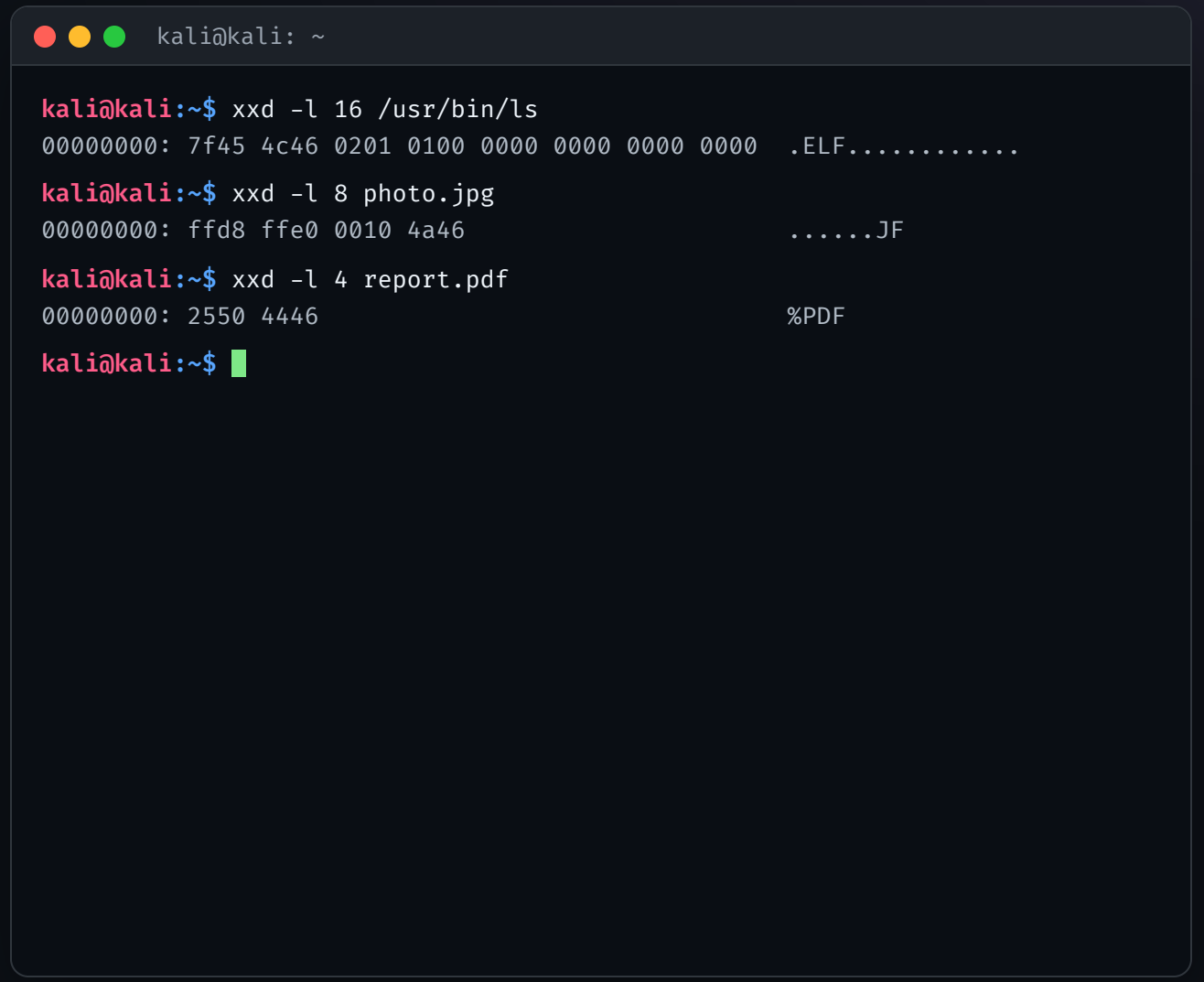
Creates a hex dump of a file (or reverses one).

```
xxd [options] file
```

| | |
|---|---|
| `-l N` | only dump N bytes |
| `-s N` | start at offset N |
| `-r` | reverse a hex dump back to binary |

**SECURITY USE**

Inspect file headers ('magic bytes') to confirm a file's true type during forensics.




▶ **Practice** Dump the first 16 bytes of a PNG and an ELF file. Compare their magic bytes.

MODULE 3 OF 6

# Users & Permissions

`whoami` `id` `sudo` `su` `chmod` `chown`

# whoami

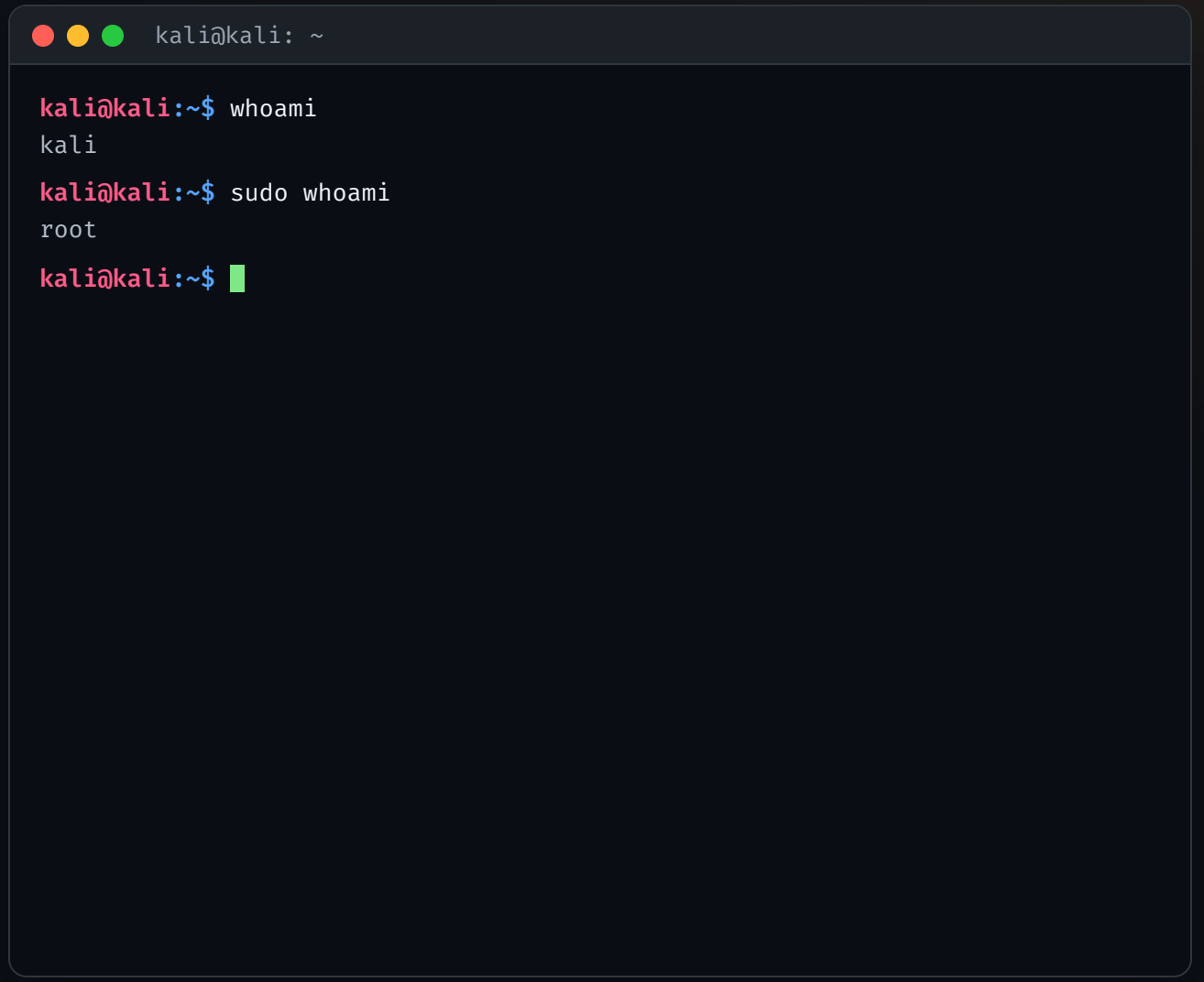
Prints the username of the current effective user.

```
whoami
```

| | |
|---|---|
| (none) | no common options; same as id -un |

**SECURITY USE**

Confirm which account you are acting as, especially after sudo or su.




▶ **Practice** Run whoami, then sudo whoami, then explain why the answers differ.

# id

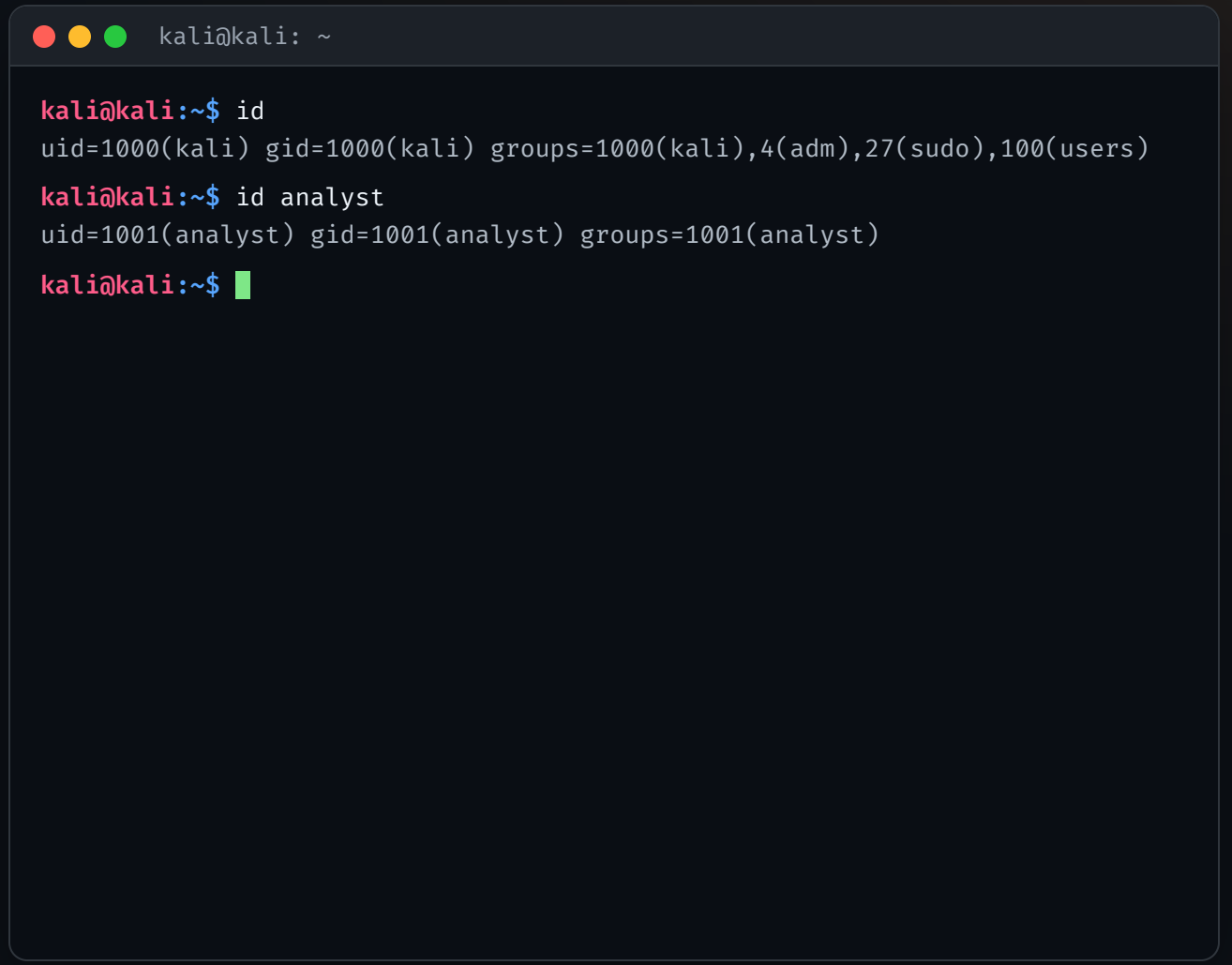
Shows user ID, group ID and all group memberships.

```
id [user]
```

| | |
|---|---|
| `-u` | user ID only |
| `-g` | primary group ID only |
| `-G` | all group IDs |
| `-n` | names instead of numbers |

**SECURITY USE**

Group membership (sudo, adm, docker) decides what an account can really do; check it in every audit.




▶ **Practice** Check which users on your system belong to the sudo group (hint: getent group sudo).

# sudo

Runs a single command with another user's (usually root's) privileges.

```
sudo [options] command
```

| | |
|---|---|
| `-l` | list what you are allowed to run |
| `-u user` | run as a specific user |
| `-i` | start a root login shell |
| `-k` | forget cached credentials |

**SECURITY USE**

Least privilege: use sudo per command, and review sudo -l output when auditing accounts.

kali@kali: ~

```
kali@kali:~$ sudo -l
User kali may run the following commands on kali:
    (ALL : ALL) ALL
kali@kali:~$ sudo grep -v '^#' /etc/sudoers | grep -v '^$' | tail -2
root    ALL=(ALL:ALL) ALL
%sudo   ALL=(ALL:ALL) ALL
kali@kali:~$
```

▶ **Practice** Run sudo -l for your account, then read /etc/sudoers with sudo less (never edit it without visudo).

# su

Switches to another user account in the current terminal.

```
su [options] [user]
```

| | |
|---|---|
| `-` | full login environment of the target user |
| `-c cmd` | run one command as that user |
| `-s shell` | use a specific shell |

**SECURITY USE**

Test what a low-privilege account can access by switching into it on your own lab system.

kali@kali: ~

```
kali@kali:~$ sudo su - analyst
analyst@kali:~$ whoami
analyst
analyst@kali:~$ cat /var/log/auth.log
cat: /var/log/auth.log: Permission denied
analyst@kali:~$ exit
kali@kali:~$
```

▶ **Practice** Create a test user (sudo adduser test1), switch to it, and see which files it cannot read.

# chmod

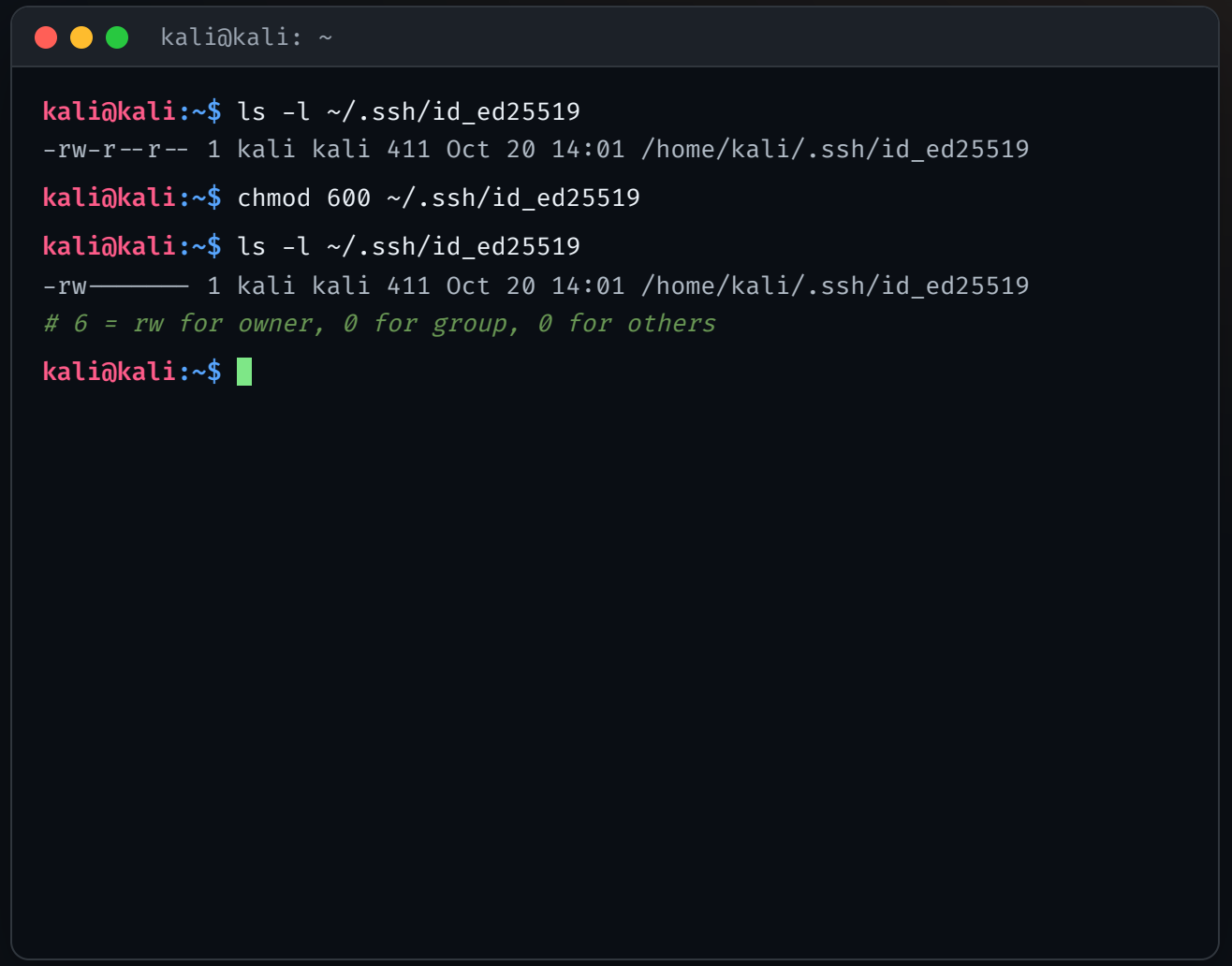
Changes file permission bits (read, write, execute).

```
chmod [options] mode file
```

| | |
|---|---|
| `u/g/o` | user, group, others |
| `+x / -w` | add execute, remove write |
| `600, 755` | octal modes |
| `-R` | recursive |

**SECURITY USE**

Lock down secrets: SSH private keys must be 600, otherwise ssh refuses to use them.




▶ **Practice** Create a script, make it executable with chmod +x, then convert its mode to octal.

# chown

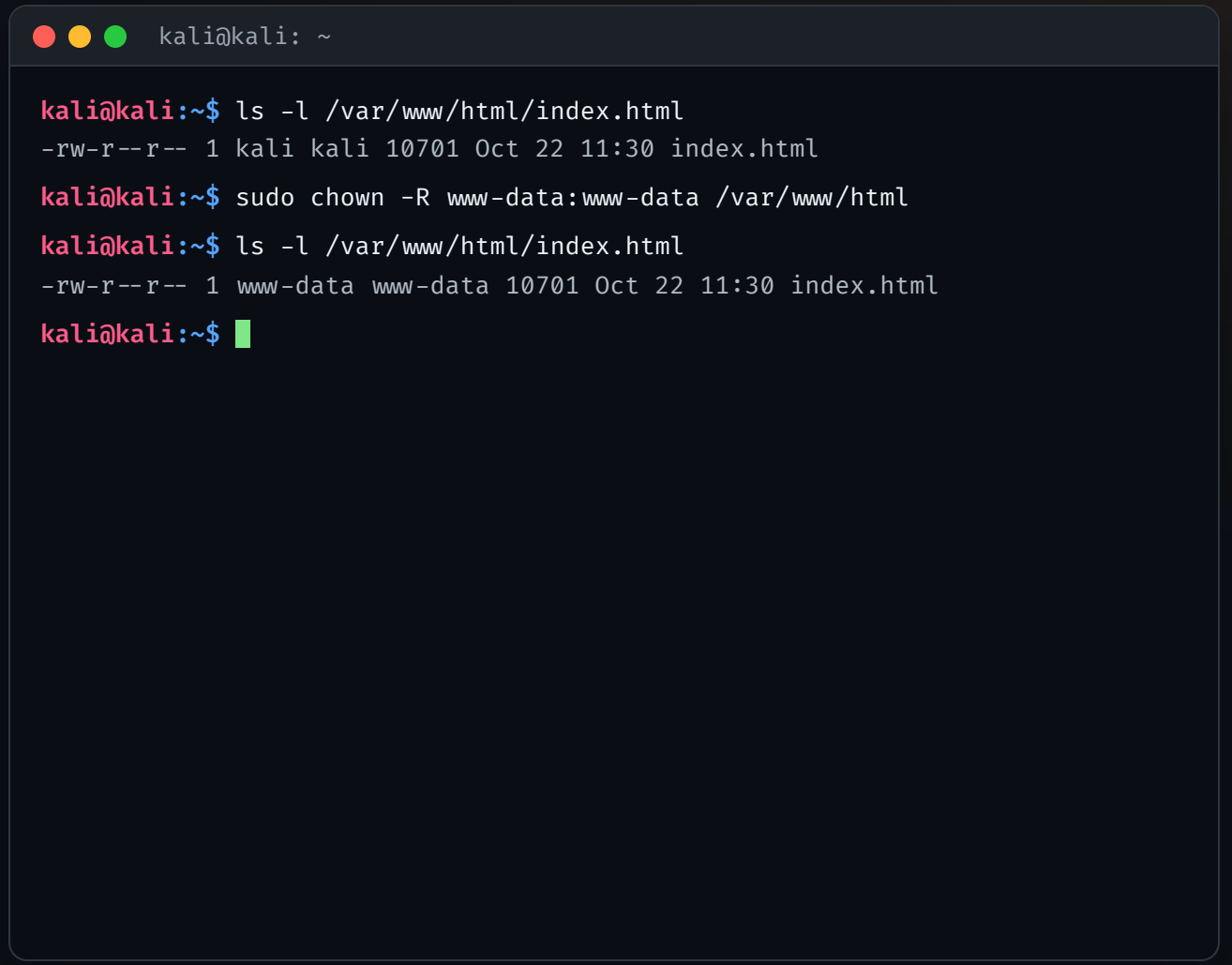
Changes the owner and group of files.

```
chown [options] user[:group] file
```

| | |
|---|---|
| `user:group` | set both owner and group |
| `-R` | recursive |
| `--reference=f` | copy ownership from file f |

**SECURITY USE**

Wrong ownership on web roots or service files is a common misconfiguration finding.




▶ **Practice** Create a file as root with sudo touch, then give it back to your user with chown.

MODULE 4 OF 6

# Processes & System

`ps` `top` `kill` `lsof` `uname` `systemctl` `journalctl` `crontab`

# ps

Takes a snapshot of running processes.

```
ps [options]
```

| | |
|---|---|
| `aux` | all processes, with user and CPU/RAM usage |
| `-ef` | full format listing |
| `--forest` | show parent/child tree |

**SECURITY USE**

Spot unexpected processes, odd parents, or programs running from /tmp.

kali@kali: ~

```
kali@kali:~$ ps aux --sort=-%cpu | head -4
USER   PID %CPU %MEM COMMAND
kali  2511 12.4  3.1 /usr/bin/firefox-esr
root   812  1.2  0.4 /usr/sbin/sshd -D
kali  2390  0.6  0.2 -zsh
kali@kali:~$ ps -ef --forest | grep -A1 sshd | head -3
root   812     1  /usr/sbin/sshd -D
root  2391   812   \_ sshd: kali [priv]
kali@kali:~$
```

▶ **Practice** List all processes owned by root, then all processes running from /tmp (if any).

# top

Shows a live, updating view of processes and resource usage.

```
top [options]
```

| | |
|---|---|
| `-u user` | only one user's processes |
| `-p PID` | watch specific PIDs |
| `P / M` | sort by CPU / memory (inside top) |
| `k` | kill a process (inside top) |

**SECURITY USE**

Sudden CPU spikes can indicate cryptominers or runaway processes; top shows them instantly.

kali@kali: ~

```
kali@kali:~$ top -b -n 1 | head -8
top - 09:20:11 up 2:14,  2 users,  load average: 0.42, 0.31, 0.25
Tasks: 214 total,   1 running, 213 sleeping
%Cpu(s):  6.1 us,  1.9 sy, 91.8 id
MiB Mem :   7951.4 total,   3120.8 free

  PID USER   PR  NI  %CPU %MEM  COMMAND
 2511 kali   20   0  12.4  3.1  firefox-esr
kali@kali:~$
```

▶ **Practice** Open top, press M to sort by memory, then press q. Note the top three processes.

28/50 PROCESSES & SYSTEM

# kill

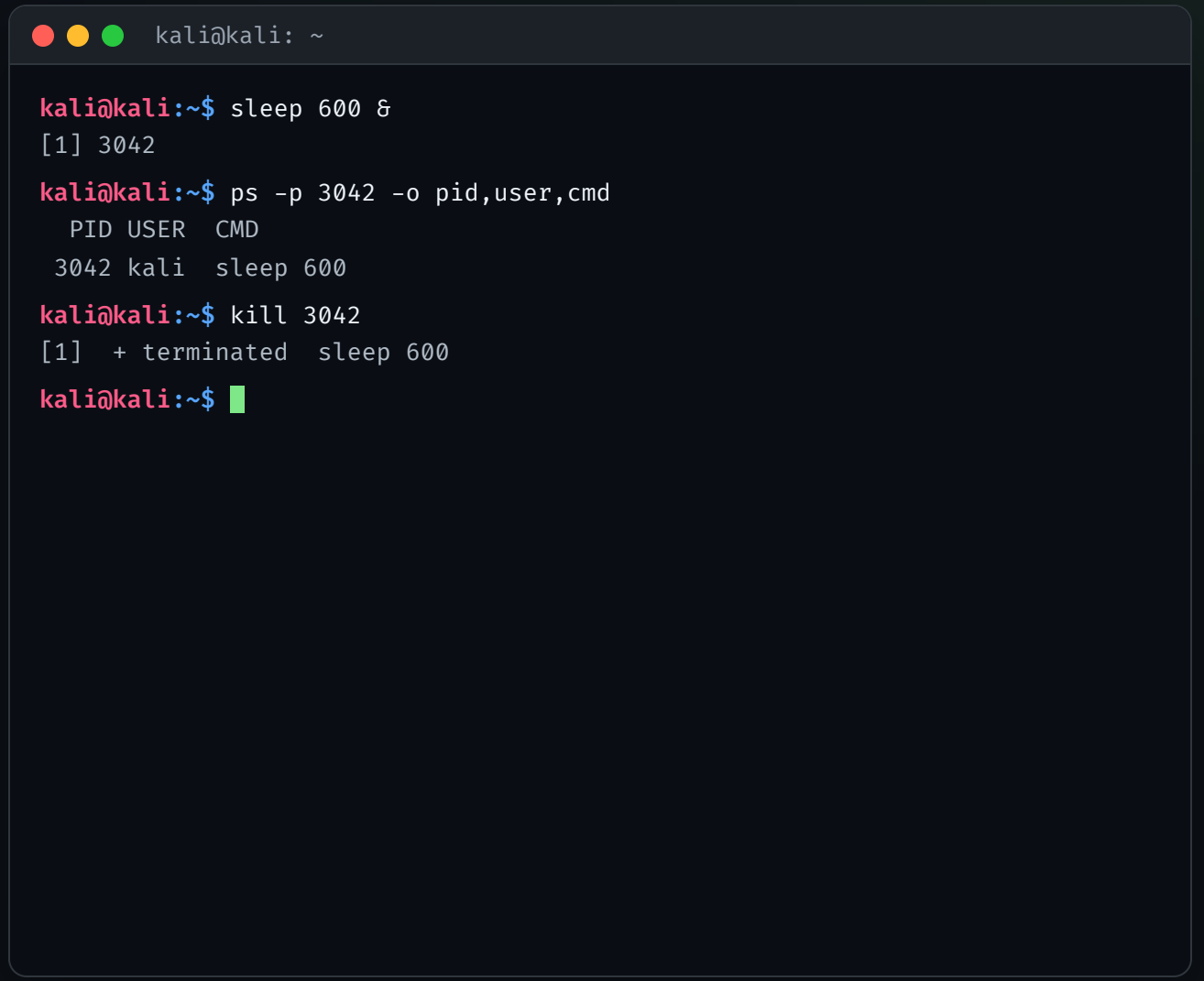
Sends a signal to a process, usually to stop it.

```
kill [-signal] PID
```

| | |
|---|---|
| `-15 (TERM)` | polite stop request (default) |
| `-9 (KILL)` | force stop, cannot be ignored |
| `-l` | list all signals |
| `pkill name` | kill by name |

**SECURITY USE**

Containment: stop a malicious or runaway process after recording its details for the report.




▶ **Practice** Start sleep 600 in the background, find its PID with ps, and stop it with kill.

29/50 PROCESSES & SYSTEM

# lsof

Lists open files: and in Linux, sockets and devices are files too.

```
lsof [options]
```

| | |
|---|---|
| `-i` | network connections |
| `-i :PORT` | who is using a port |
| `-p PID` | files opened by a process |
| `-u user` | files opened by a user |

**SECURITY USE**

Map a listening port back to the exact process and binary that opened it.

kali@kali: ~

```
kali@kali:~$ sudo lsof -i :22
COMMAND  PID USER  FD  TYPE  NODE NAME
sshd     812 root  3u  IPv4  TCP  *:ssh (LISTEN)
sshd    2391 root  4u  IPv4  TCP  kali:ssh→192.168.56.1:51522 (ESTABLISHED)
kali@kali:~$ sudo lsof -p 812 | grep txt
sshd 812 root txt REG 8,1 917192 /usr/sbin/sshd
kali@kali:~$
```

▶ **Practice** Find which process is listening on each open port of your machine with lsof -i -P -n.

# uname

Prints system information such as kernel version and architecture.

```
uname [options]
```

| | |
|---|---|
| `-a` | all information |
| `-r` | kernel release |
| `-m` | machine architecture |

**SECURITY USE**

Kernel version tells you which security patches are present when you assess a host.

kali@kali: ~

```
kali@kali:~$ uname -a
Linux kali 6.11.2-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.11.2-1kali1 x86_64 GN
U/Linux
kali@kali:~$ uname -r
6.11.2-amd64
kali@kali:~$ cat /etc/os-release | head -2
PRETTY_NAME="Kali GNU/Linux Rolling"
NAME="Kali GNU/Linux"
kali@kali:~$
```

▶ **Practice** Record your kernel release and compare it with the latest available version after apt update.

# systemctl

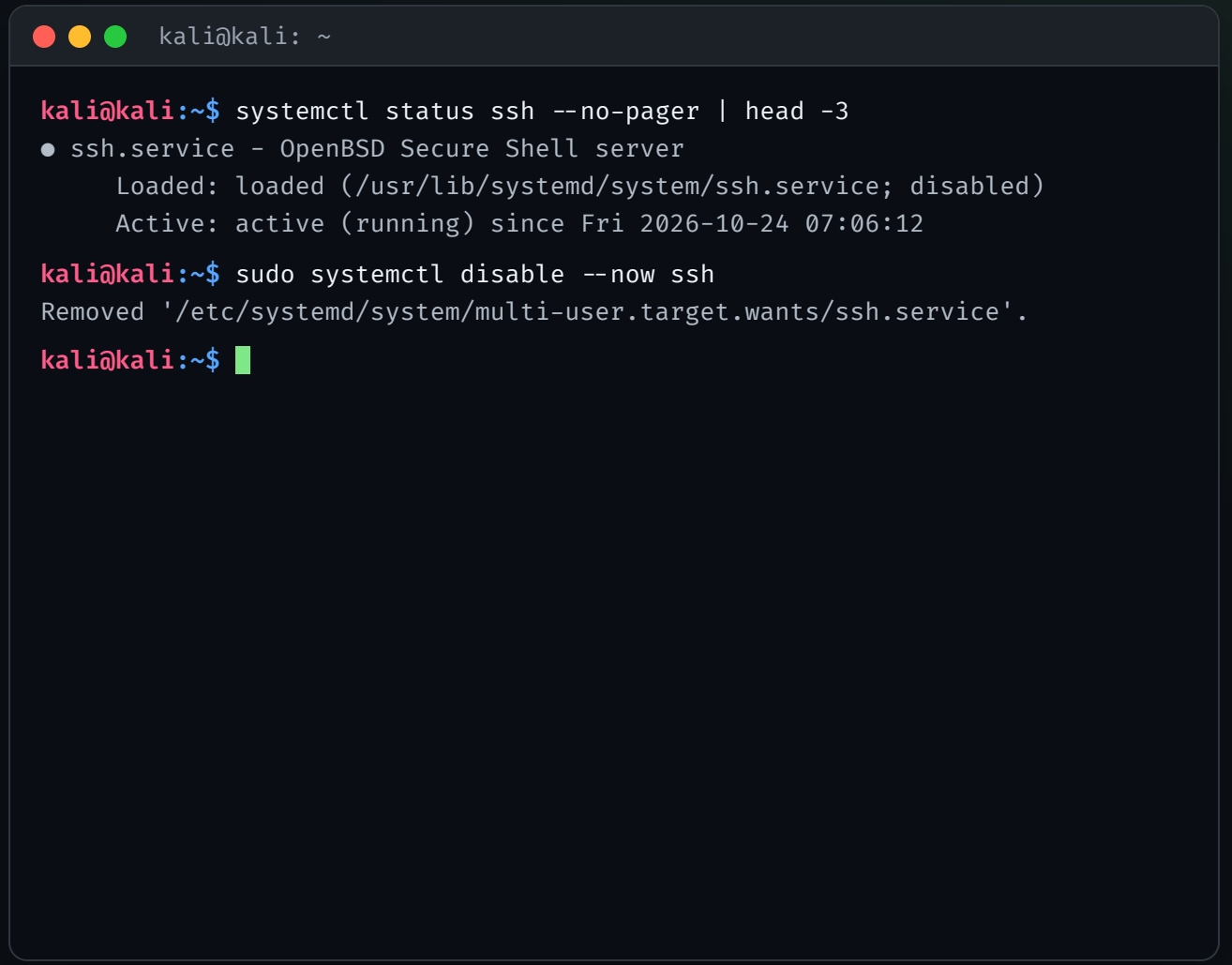
Controls systemd services: start, stop, enable and inspect them.

```
systemctl [command] [unit]
```

| | |
|---|---|
| `status` | is it running? recent logs |
| `start / stop` | run or halt now |
| `enable / disable` | start at boot or not |
| `list-units --type=service` | list services |

**SECURITY USE**

Reduce attack surface: disable services you do not need and check for unknown ones.




▶ **Practice** List running services with systemctl list-units --type=service --state=running. Do you need them all?

# journalctl

Queries the systemd journal (system and service logs).

```
journalctl [options]
```

| | |
|---|---|
| `-u unit` | logs for one service |
| `-f` | follow live |
| `--since` | time filter, e.g. '1 hour ago' |
| `-p err` | only errors and worse |

**SECURITY USE**

Central place to review service activity, failed logins and errors with precise time filters.

kali@kali: ~

```
kali@kali:~$ journalctl -u ssh --since '1 hour ago' --no-pager | tail -3
Oct 24 08:51:05 kali sshd[2210]: Failed password for root from 192.168.56.23
port 50418
Oct 24 08:51:09 kali sshd[2210]: Connection closed by 192.168.56.23 port 504
18
Oct 24 09:14:10 kali sshd[2391]: Accepted publickey for kali from 192.168.5
6.1
kali@kali:~$ journalctl -p err -b --no-pager | wc -l
4
kali@kali:~$
```

▶ **Practice** Show all error-level messages since the last boot with journalctl -p err -b.

# crontab

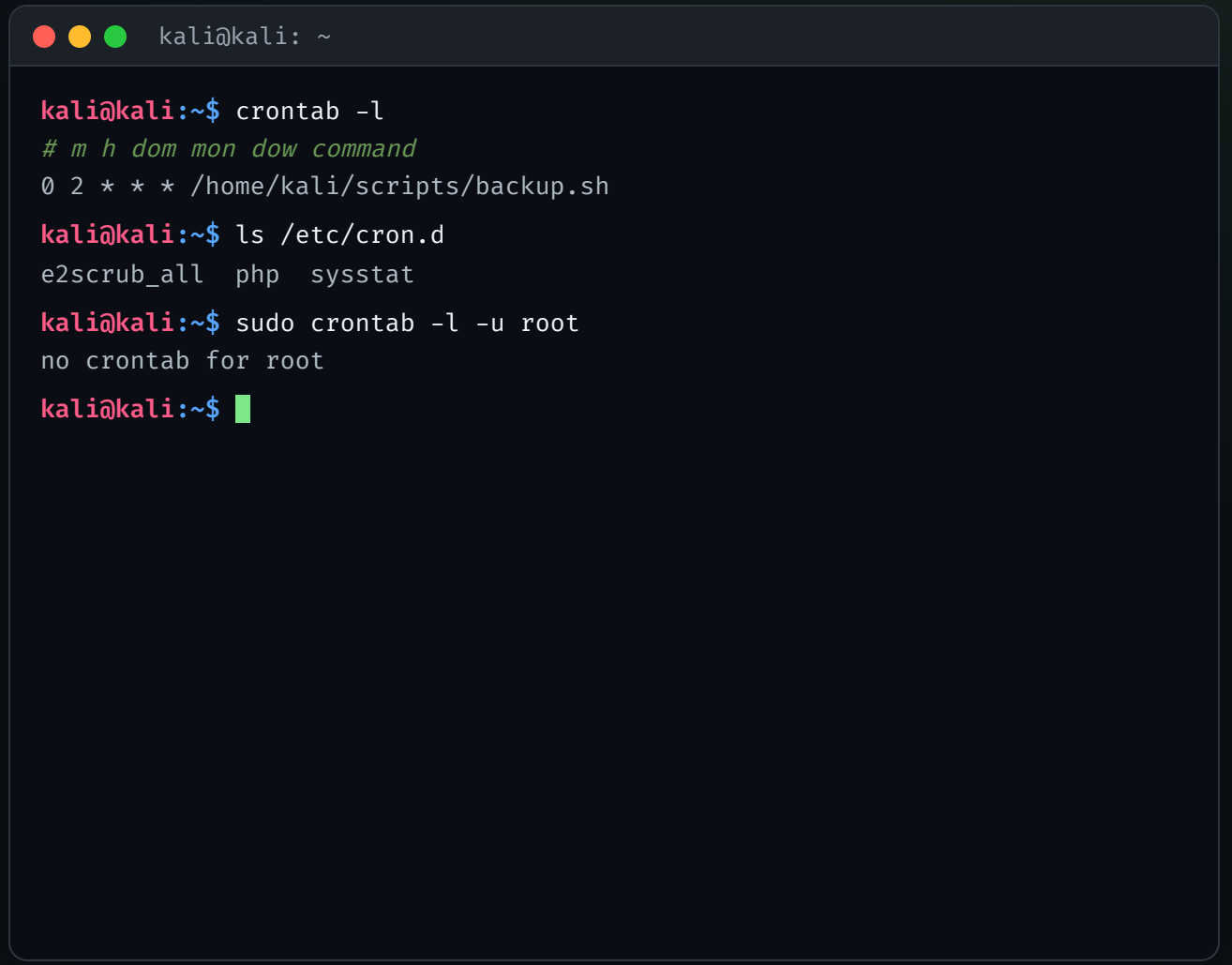
Views and edits scheduled tasks (cron jobs).

```
crontab [options]
```

| | |
|---|---|
| `-l` | list your cron jobs |
| `-e` | edit your cron jobs |
| `-u user` | act on another user (root) |
| `/etc/cron.*` | system-wide schedules |

**SECURITY USE**

Scheduled tasks are a common persistence spot; review every user's crontab and /etc/cron.d.




▶ **Practice** List your crontab and the contents of /etc/cron.d. Can you explain every entry?

MODULE 5 OF 6

# Networking

`ip` `ss` `ping` `traceroute` `dig` `curl` `wget` `nc` `ssh` `scp` `nmap` `tcpdump`

34/50 NETWORKING

# ip

Shows and configures network interfaces, addresses and routes.

```
ip [object] [command]
```

| | |
|---|---|
| `ip a` | addresses on every interface |
| `ip r` | routing table |
| `ip link` | interface state |
| `ip neigh` | ARP / neighbour table |

**SECURITY USE**

Know your own network position first: your IP, gateway and neighbours on the segment.

kali@kali: ~

```
kali@kali:~$ ip -br a
lo        UNKNOWN  127.0.0.1/8
eth0      UP       192.168.56.101/24
kali@kali:~$ ip r
default via 192.168.56.1 dev eth0
192.168.56.0/24 dev eth0 proto kernel src 192.168.56.101
kali@kali:~$ ip neigh
192.168.56.1 dev eth0 lladdr 0a:00:27:00:00:00 REACHABLE
kali@kali:~$
```

▶ **Practice** Find your IP address, default gateway and the MAC address of the gateway.

35/50 NETWORKING

# ss

Displays sockets: listening ports and active connections (modern netstat).

```
ss [options]
```

| | |
|---|---|
| `-t / -u` | TCP / UDP |
| `-l` | listening sockets only |
| `-n` | numeric, no name lookup |
| `-p` | show the owning process |

**SECURITY USE**

Audit what your host exposes: every listening port should be expected and documented.




▶ **Practice** List your listening ports with ss -tulnp. Which ones are reachable from other machines (0.0.0.0)?

36/50 NETWORKING

# ping

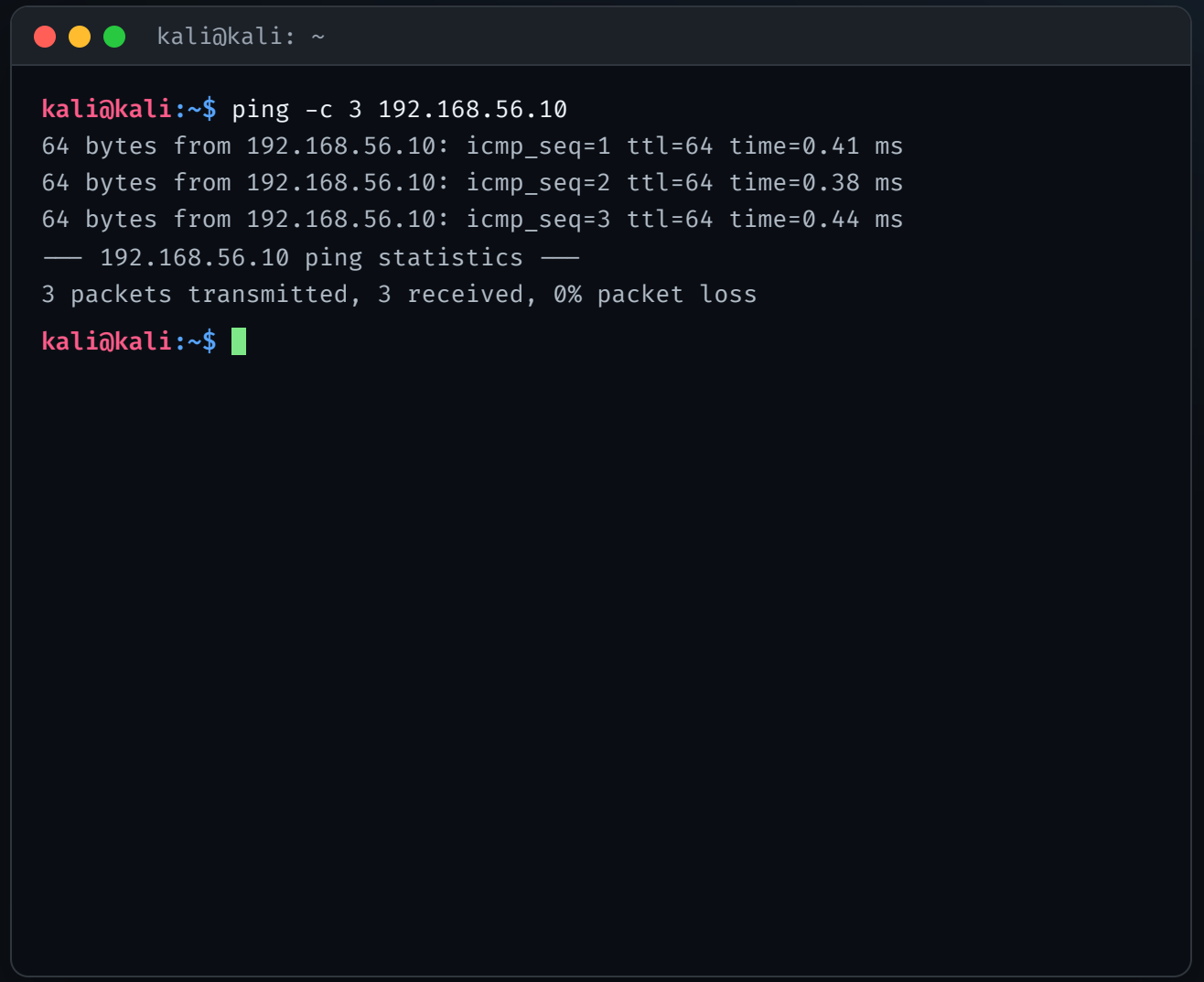
Tests reachability and round-trip time using ICMP echo.

```
ping [options] host
```

| | |
|---|---|
| `-c N` | send N packets then stop |
| `-i sec` | interval between packets |
| `-W sec` | timeout per reply |

**SECURITY USE**

First connectivity check in any lab; also shows whether ICMP is filtered by a firewall.




▶ **Practice** Ping your gateway 4 times and note the average latency and the TTL value.

37/50 NETWORKING

# traceroute

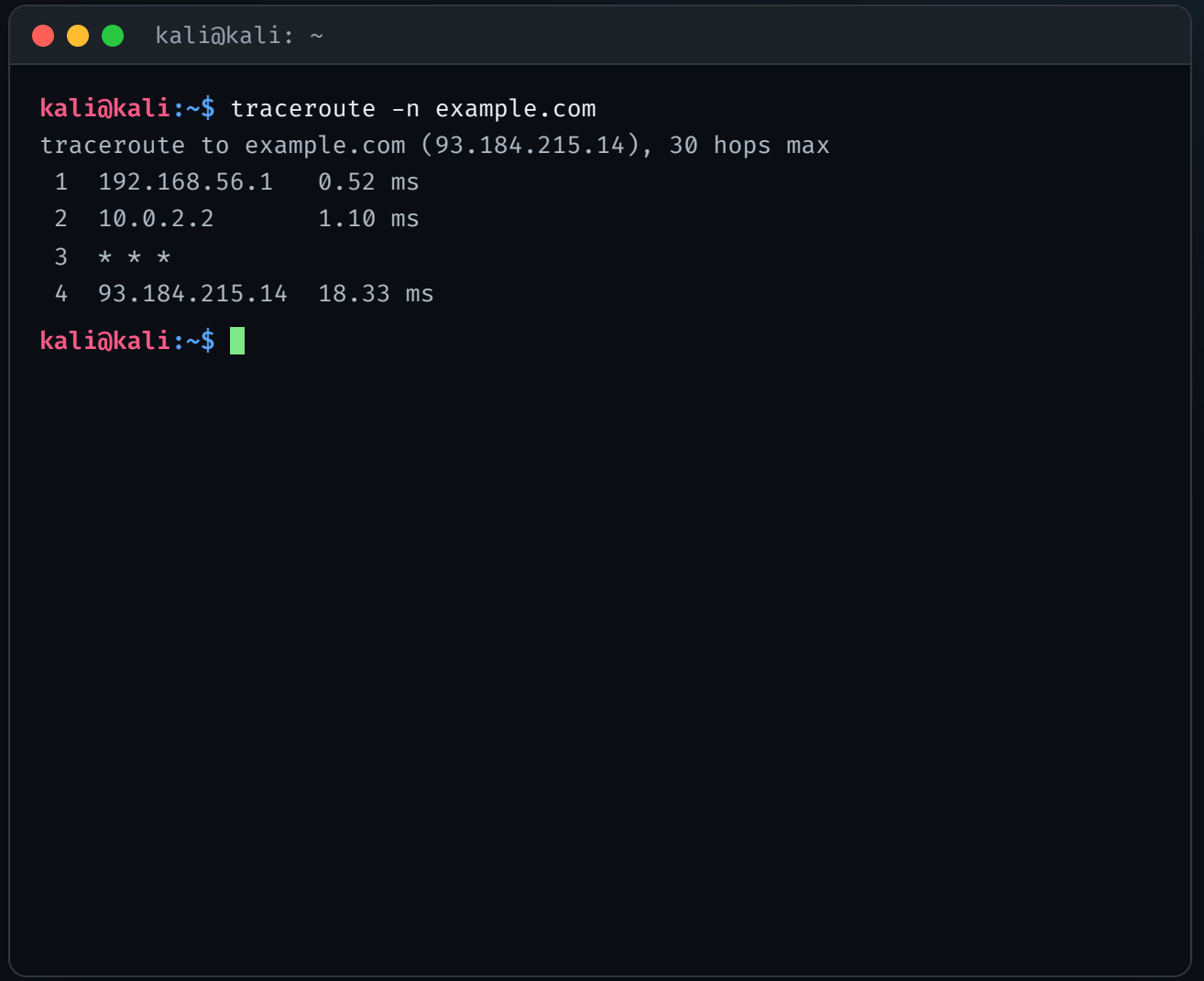
Shows each router hop between you and a destination.

```
traceroute [options] host
```

| | |
|---|---|
| `-n` | no DNS lookups (faster) |
| `-I` | use ICMP instead of UDP |
| `-T` | use TCP SYN (often passes firewalls) |

**SECURITY USE**

Understand network paths and where traffic is filtered or slowed.




▶ **Practice** Trace the route to example.com. At which hop does your local network end?

38/50 NETWORKING

# dig

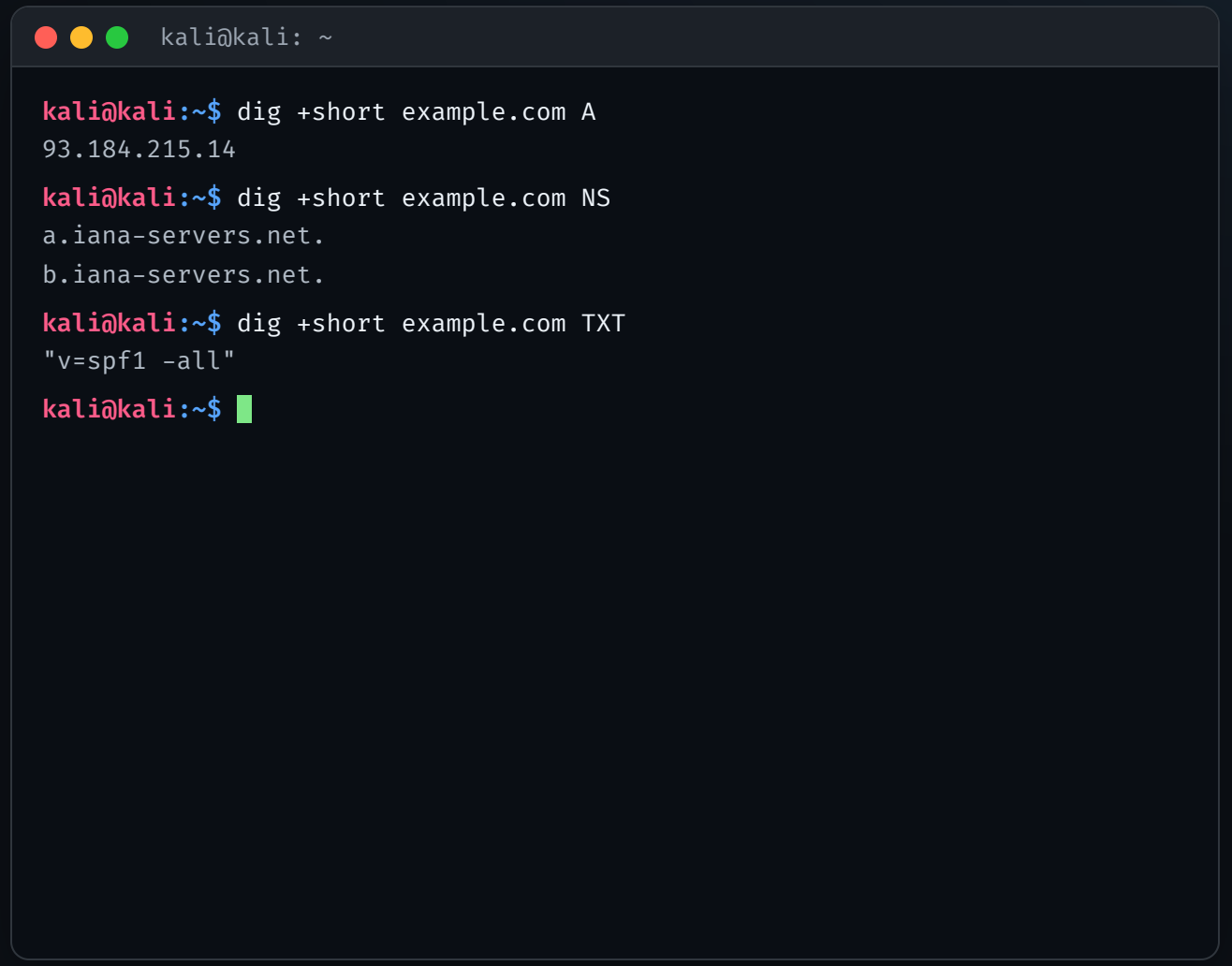
Queries DNS servers for records (A, MX, TXT, NS, ...).

```
dig [@server] name [type]
```

| | |
|---|---|
| `+short` | answers only |
| `MX / TXT / NS` | record type |
| `-x IP` | reverse lookup |
| `@8.8.8.8` | ask a specific resolver |

**SECURITY USE**

DNS reconnaissance of your own domains, and checking SPF/DMARC records for email security.




▶ **Practice** Look up the MX and TXT records of a domain you own or your school's domain.

39/50 NETWORKING

# curl

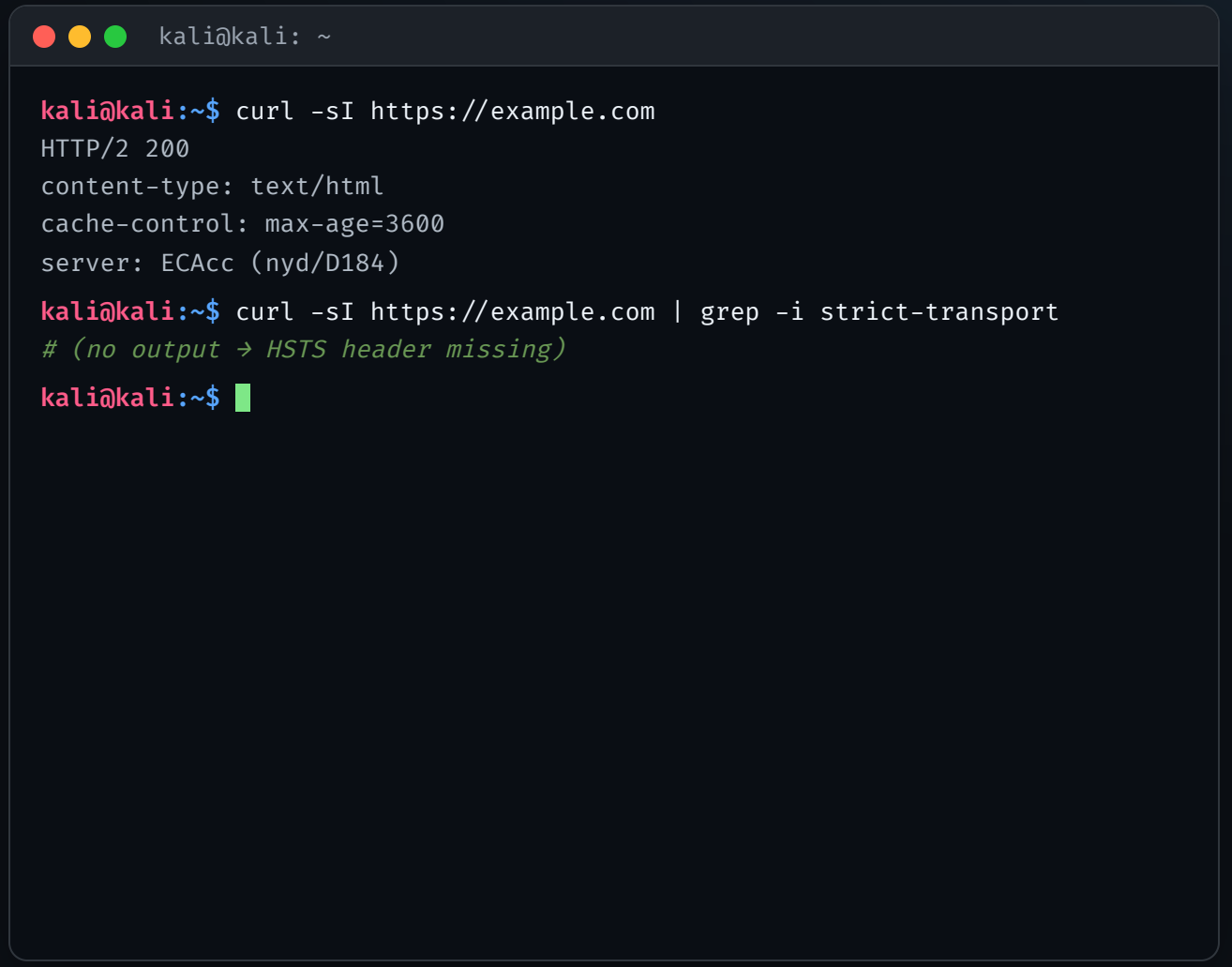
Transfers data to or from a URL; ideal for inspecting HTTP.

```
curl [options] URL
```

| | |
|---|---|
| `-I` | fetch headers only |
| `-v` | verbose: show the whole exchange |
| `-L` | follow redirects |
| `-o file` | save to file |

**SECURITY USE**

Check security headers (HSTS, CSP, X-Frame-Options) and server banners on your own sites.




▶ **Practice** Inspect the response headers of a site you own. Which security headers are missing?

40/50 NETWORKING

# wget

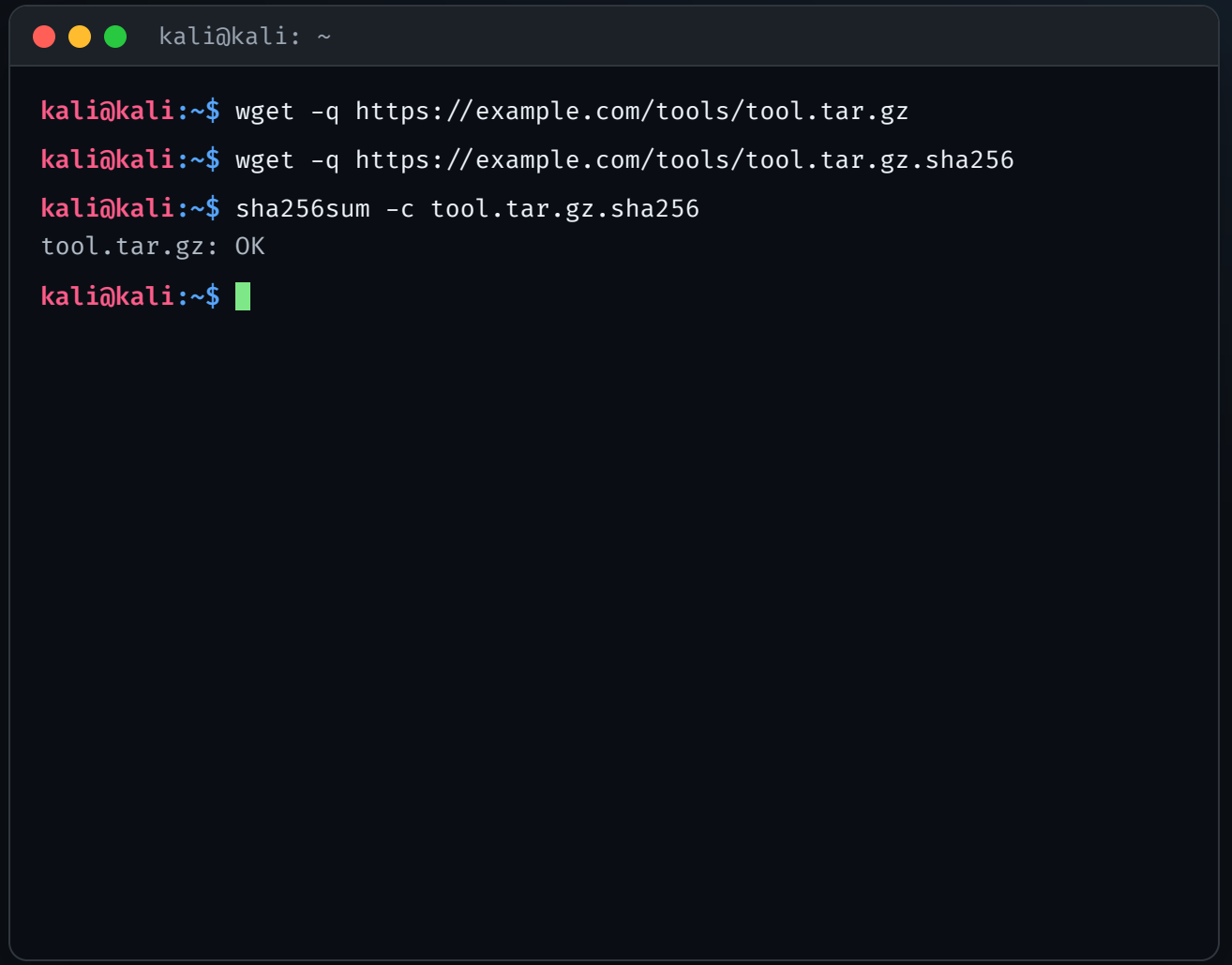
Downloads files from the web, non-interactively.

```
wget [options] URL
```

| | |
|---|---|
| `-O file` | save under a chosen name |
| `-c` | resume a partial download |
| `-q` | quiet |
| `--spider` | check the URL exists, download nothing |

**SECURITY USE**

Fetch tools and wordlists, then verify their checksums before use.




▶ **Practice** Download any file with wget, then use --spider to check a URL without downloading.

# nc

Netcat: reads and writes raw TCP/UDP connections.

```
nc [options] host port
```

| | |
|---|---|
| `-z` | scan without sending data |
| `-v` | verbose |
| `-l` | listen mode |
| `-w sec` | timeout |

**SECURITY USE**

Quick check of whether a port is open, and simple banner checks on your own services.

kali@kali: ~

```
kali@kali:~$ nc -zv 192.168.56.10 22 80 443
Connection to 192.168.56.10 22 port [tcp/ssh] succeeded!
Connection to 192.168.56.10 80 port [tcp/http] succeeded!
nc: connect to 192.168.56.10 port 443 (tcp) failed: Connection refused
kali@kali:~$ nc -v 192.168.56.10 22
SSH-2.0-OpenSSH_9.9p1 Debian-1
^C
kali@kali:~$
```

▶ **Practice** On two lab VMs: start nc -l 4444 on one and connect to it from the other. Type a message.

# ssh

Secure Shell: encrypted remote login and command execution.

```
ssh [options] user@host
```

| | |
|---|---|
| `-i key` | use a specific private key |
| `-p port` | non-default port |
| `-L` | local port forward |
| `-v` | verbose debug |

**SECURITY USE**

Secure administration. Prefer key-based login and disable password and root login on servers.

kali@kali: ~

```
kali@kali:~$ ssh-keygen -t ed25519 -C 'kali@lab'
Your public key has been saved in /home/kali/.ssh/id_ed25519.pub
kali@kali:~$ ssh-copy-id analyst@192.168.56.10
Number of key(s) added: 1
kali@kali:~$ ssh analyst@192.168.56.10
analyst@lab-web:~$
kali@kali:~$
```

▶ **Practice** Create an ed25519 key pair and log into a lab VM without a password.

# scp

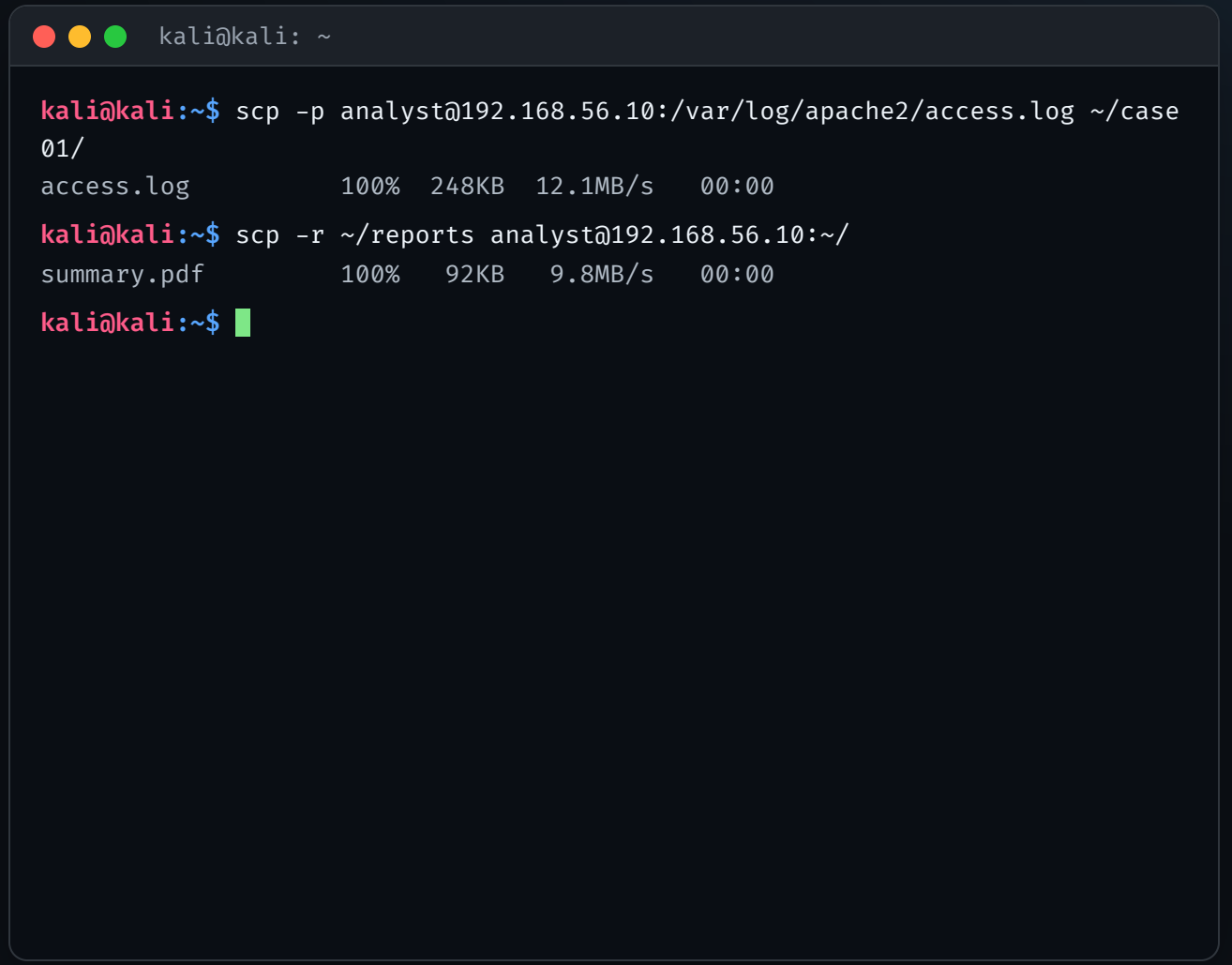
Copies files securely between hosts over SSH.

```
scp [options] source dest
```

| | |
|---|---|
| `-r` | copy directories |
| `-P port` | SSH port (capital P) |
| `-i key` | identity file |
| `-p` | preserve times and modes |

**SECURITY USE**

Move evidence or logs off a host over an encrypted channel.




▶ **Practice** Copy a file from your Kali VM to another lab VM, then copy it back with a new name.

# nmap

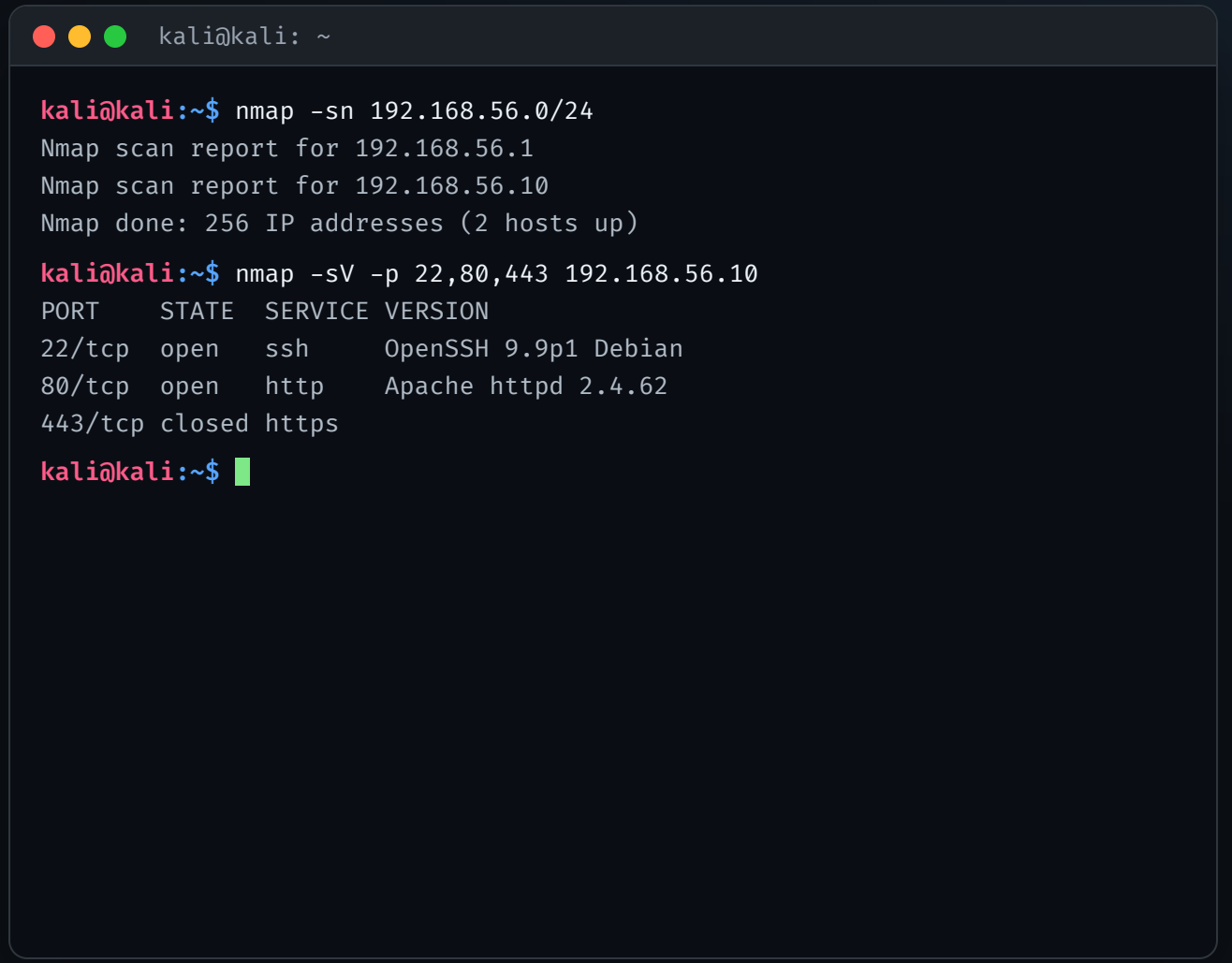
Network mapper: discovers hosts, open ports and service versions.

```
nmap [options] target
```

| | |
|---|---|
| `-sn` | host discovery only (no port scan) |
| `-sV` | detect service versions |
| `-p` | choose ports (e.g. -p 1-1000) |
| `-oN file` | save output |

**SECURITY USE**

Inventory your own network and verify firewall rules. Only scan systems you own or are authorised to test.




▶ **Practice** Scan your own lab VM or scanme.nmap.org (explicitly allowed) and compare with ss -tulnp on the target.

45/50 NETWORKING

# tcpdump

Captures and inspects network packets from the command line.

```
tcpdump [options] [filter]
```

| | |
|---|---|
| `-i iface` | interface to capture on |
| `-n` | no name resolution |
| `-w file` | write a .pcap for Wireshark |
| `port 53` | BPF filter expression |

**SECURITY USE**

See exactly what crosses the wire: troubleshoot, verify encryption, and record evidence.

kali@kali: ~

```
kali@kali:~$ sudo tcpdump -i eth0 -n -c 3 port 53
listening on eth0, link-type EN10MB (Ethernet)
09:31:02.114 IP 192.168.56.101.40122 > 192.168.56.1.53: A? example.com.
09:31:02.131 IP 192.168.56.1.53 > 192.168.56.101.40122: A 93.184.215.14
3 packets captured
kali@kali:~$ sudo tcpdump -i eth0 -w lab.pcap
# open lab.pcap later in Wireshark
kali@kali:~$
```

▶ **Practice** Capture 20 packets while running dig example.com, and identify the DNS query and answer.

MODULE 6 OF 6

# Security & Archives

`iptables` `tar` `base64` `sha256sum` `openssl`

# iptables

Configures the Linux kernel packet-filtering firewall.

```
iptables [-t table] command [rule]
```

| | |
|---|---|
| `-L -n -v` | list rules with counters |
| `-A / -I` | append / insert a rule |
| `-j ACCEPT\|DROP` | rule action |
| `-P` | set the chain's default policy |

**SECURITY USE**

Host hardening: allow only the services you need and drop the rest. (nftables/ufw are modern front-ends.)

kali@kali: ~

```
kali@kali:~$ sudo iptables -A INPUT -i lo -j ACCEPT
kali@kali:~$ sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
kali@kali:~$ sudo iptables -A INPUT -p tcp --dport 22 -s 192.168.56.0/24 -j ACCEPT
kali@kali:~$ sudo iptables -P INPUT DROP
kali@kali:~$ sudo iptables -L INPUT -n --line-numbers
1  ACCEPT  all  --  0.0.0.0/0          0.0.0.0/0
2  ACCEPT  all  --  0.0.0.0/0          0.0.0.0/0   state RELATED,ESTABLISHED
3  ACCEPT  tcp  --  192.168.56.0/24    0.0.0.0/0   tcp dpt:22
kali@kali:~$
```

▶ **Practice** On a lab VM, allow SSH only from your subnet, then verify with nmap from inside and outside it.

# tar

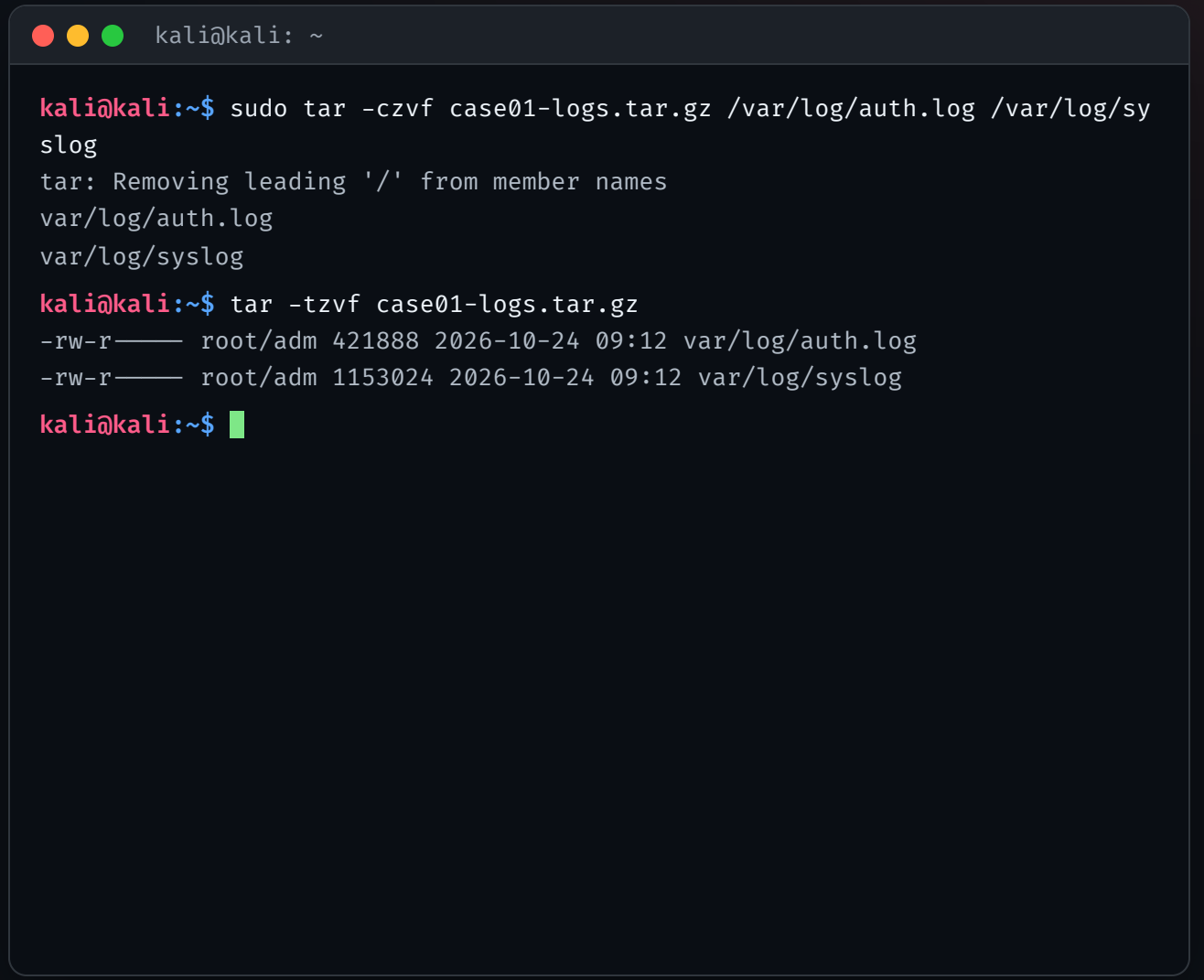
Bundles files into archives and extracts them, with optional compression.

```
tar [options] archive [files]
```

| | |
|---|---|
| `-c / -x` | create / extract |
| `-z / -J` | gzip / xz compression |
| `-v` | verbose |
| `-f file` | archive filename |
| `-t` | list contents |

**SECURITY USE**

Package collected logs as one evidence bundle; always list (-t) an unknown archive before extracting.




▶ **Practice** Archive a folder with tar -czf, list it with -t, and extract it into a new directory.

# base64

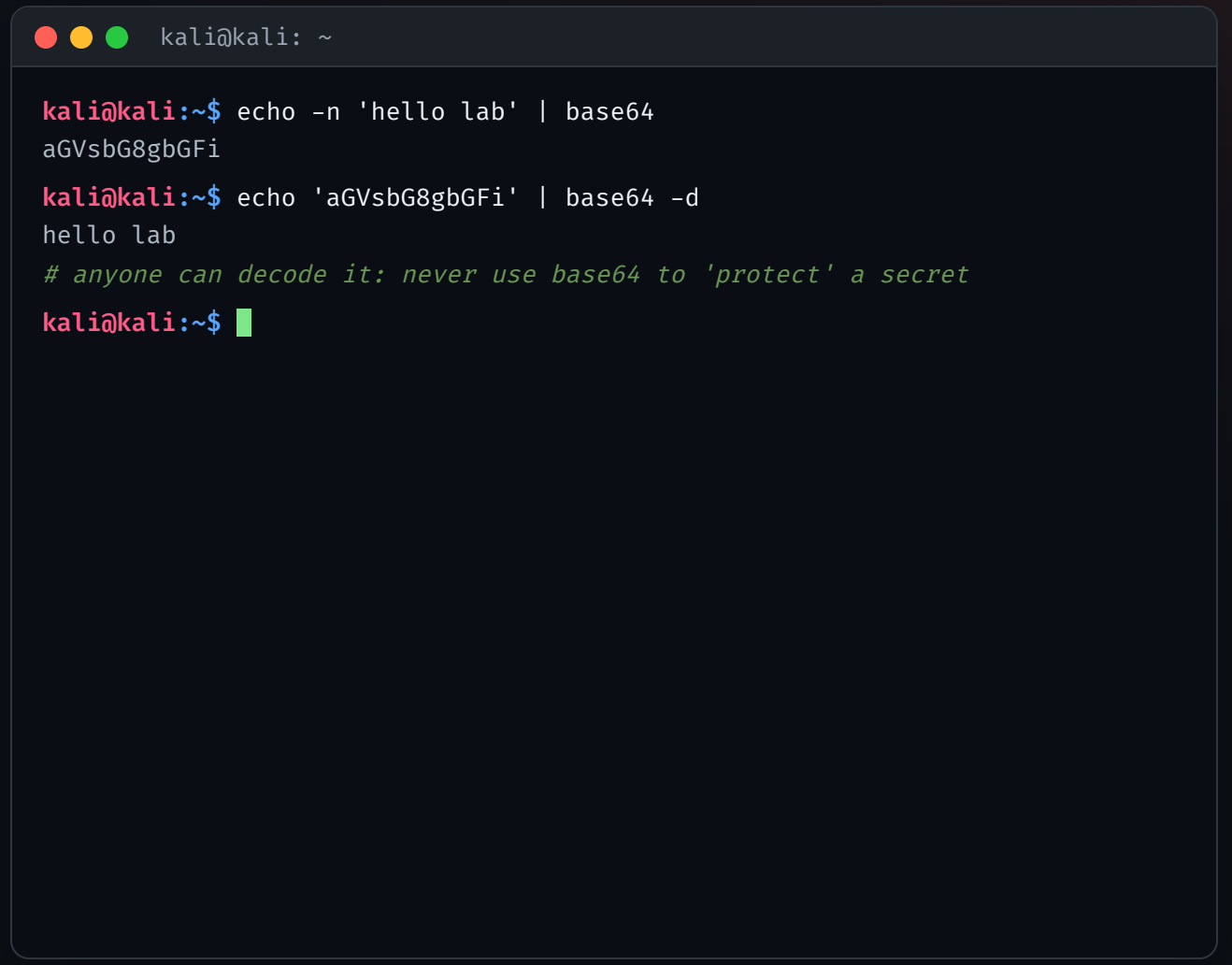
Encodes or decodes data in Base64 text form.

```
base64 [options] [file]
```

| | |
|---|---|
| `-d` | decode |
| `-w 0` | no line wrapping |

**SECURITY USE**

Base64 is encoding, NOT encryption. Analysts decode it constantly when reviewing logs and scripts.




▶ **Practice** Encode your name in Base64, then decode it. Why is this not a way to hide passwords?

# sha256sum

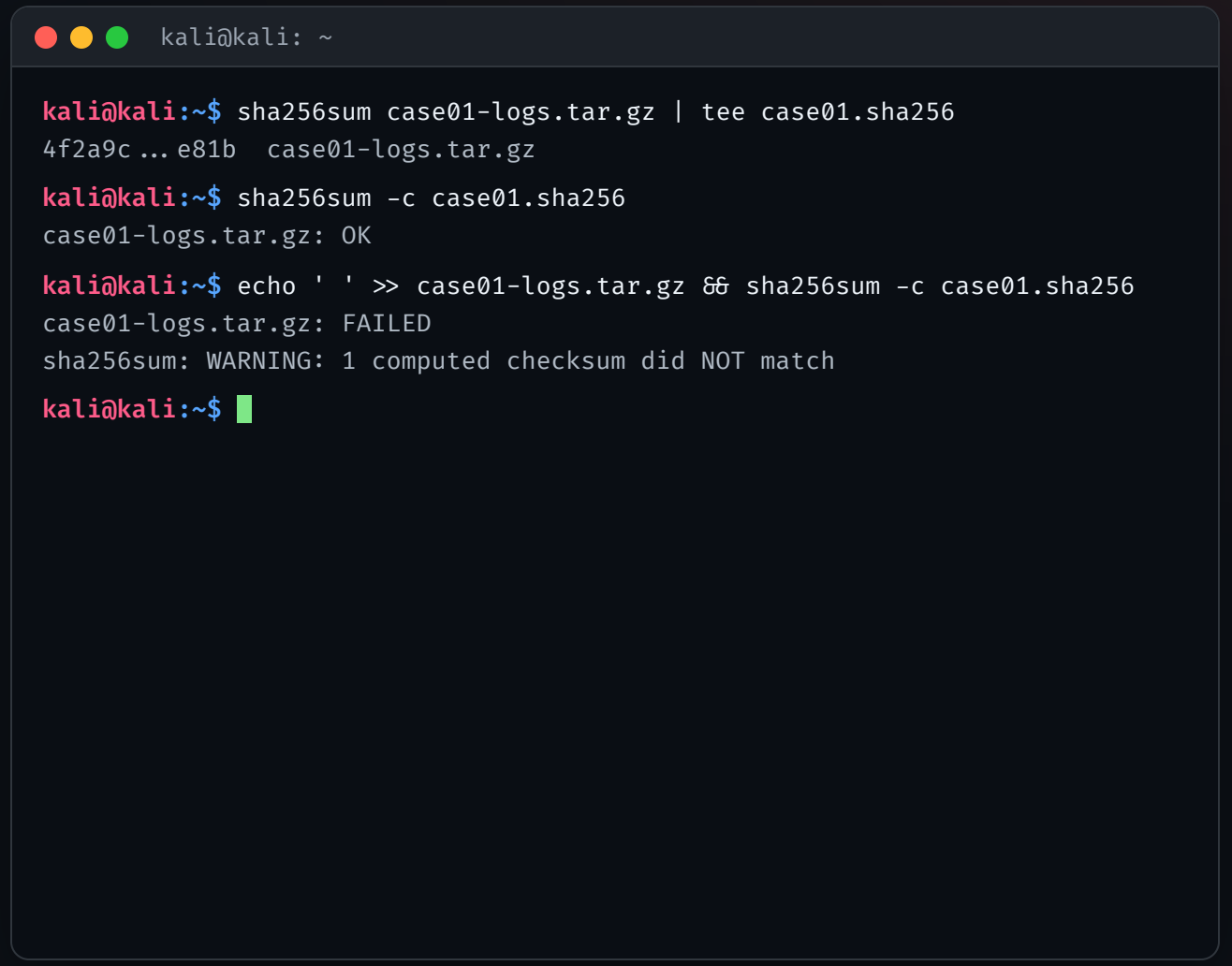
Computes and verifies SHA-256 cryptographic hashes.

```
sha256sum [options] file
```

| | |
|---|---|
| `-c file` | check hashes listed in a file |
| `--quiet` | only report failures |
| `md5sum / sha1sum` | older (weaker) siblings |

**SECURITY USE**

Verify downloads, prove evidence integrity, and look up file hashes in threat-intel databases.




▶ **Practice** Hash a file, change one character in it, and hash it again. How much of the hash changes?

# openssl

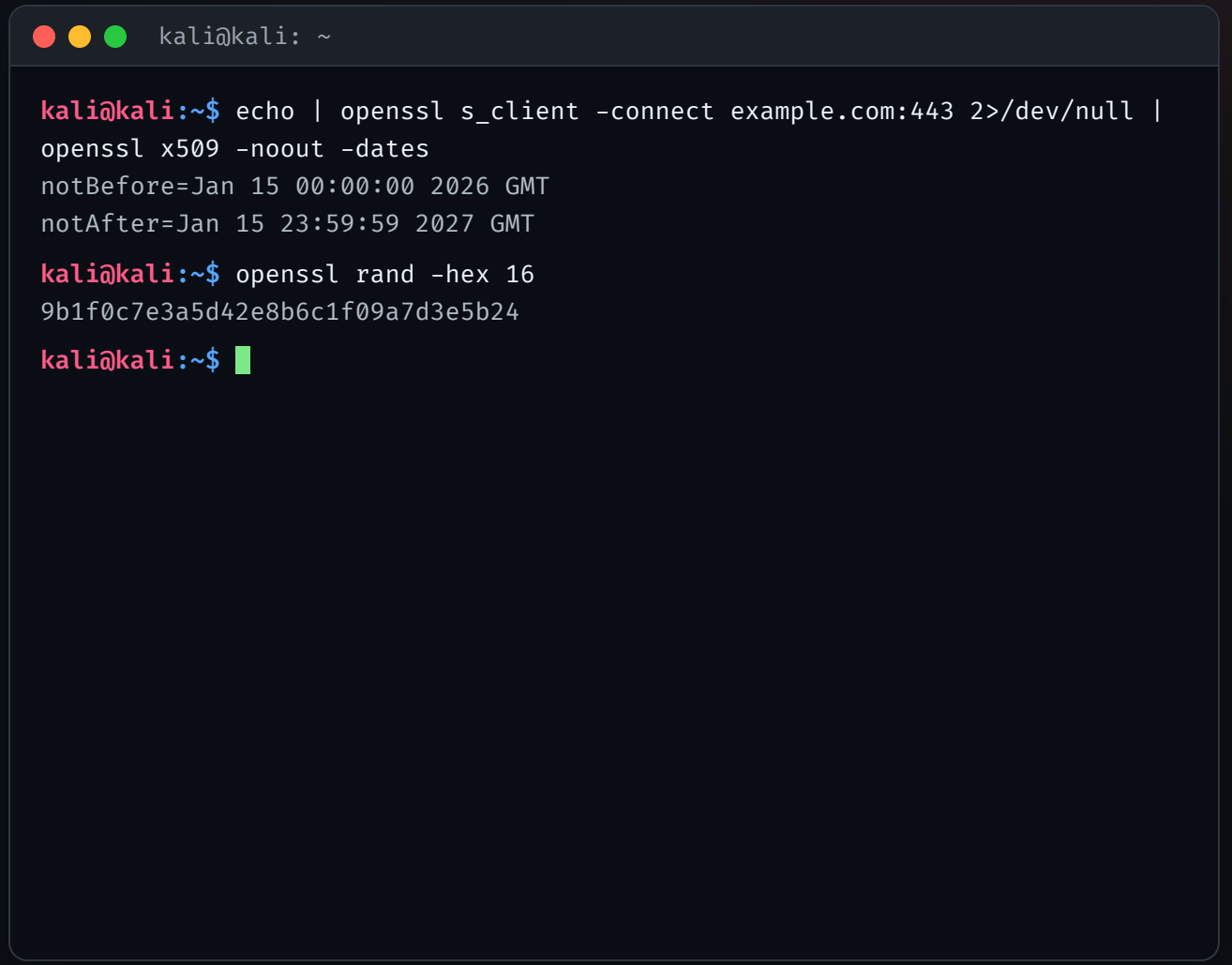
A toolkit for TLS, certificates, hashing and encryption.

```
openssl [command] [options]
```

| | |
|---|---|
| `s_client -connect` | inspect a server's TLS |
| `x509 -text` | read certificate details |
| `rand -hex N` | generate random bytes |
| `enc -aes-256-cbc` | encrypt a file |

**SECURITY USE**

Check certificate expiry and TLS configuration of your services; generate strong random secrets.




▶ **Practice** Check the certificate expiry date of a website you use daily with openssl s_client.

FINAL LAB

# Put it together: a mini investigation

1. Who has logged in, and who failed? `grep · awk · sort · uniq`
2. What is listening, and which process owns it? `ss · lsof · ps`
3. Anything scheduled or recently changed? `crontab · find`
4. Package the evidence and prove its integrity. `tar · sha256sum`
5. Harden: disable unused services, firewall the rest. `systemctl · iptables`

case01

```
kali@kali:~$ sudo grep 'Failed password' /var/log/auth.log \
    | awk '{print $(NF-3)}' | sort | uniq -c | sort -rn | head -3
     29 192.168.56.23
kali@kali:~$ sudo ss -tulnp | grep LISTEN | wc -l
3
kali@kali:~$ find /etc -mtime -1 -type f
/etc/hosts
kali@kali:~$ sudo tar -czf case.tgz /var/log/auth.log && sha256sum case.tgz
4f2a9c … e81b  case.tgz
kali@kali:~$
```

RECAP

# Keep practising

Read the manual for any command with `man <command>` or `<command> --help`. Repeat each practice task until you can do it without the slide.

Next steps: shell scripting · log analysis with journalctl · Wireshark · hardening benchmarks (CIS)

